



GIM6010-8 Micro Motor User Manual

Table of Contents

1	Motor Specifications	5
1.1	Drawings and Dimensions	5
1.2	Electrical Characteristics	6
1.3	Mechanical Characteristics	6
2	Driver Information	8
2.1	Appearance and 3D Dimensions	8
2.2	Interface Overview	9
2.3	Specifications	9
2.4	Detailed Interface Definitions	10
2.5	Primary Components and Specifications	14
3	Testing and Calibration Instructions	15
3.1	Getting Started Guide	15
3.2	Firmware Update Download	39
4	Communication Protocols and Examples	41
4.1	CAN SIMPLE Protocol	41
4.2	CANOPEN Protocol	54
4.3	RS485 Protocol (MODBUS RTU)	56
4.4	PWM Input Control	81
4.5	Pulse/Direction Control	83
4.6	SDK	85
5	Frequently Asked Questions and Error Codes (To Be Updated)	95
5.6	Frequently Asked Questions (FAQ)	95
5.7	Error Codes	95

Version History

Version Number	Date	Revision/Description
0.1	December 5, 2023	First version supporting odrivetool
0.2	December 15, 2023	Added command list and command categorization
0.3	2023.12.19	Added Python, Arduino, and ROS SDK support
0.4	2023.12.23	Added description of USB and CAN compatibility switching
0.5	2023.12.29	Added practical case studies for host computer debugging
0.6	2024.1.2	Added CAN protocol and Python debugging practical case studies
0.7	January 8, 2024	Add CAN interface 120R matching resistor switch option
0.8	February 15, 2024	Added second encoder-related content and user zero-point setting
0.9	February 23, 2024	Added CAN commands for modifying parameters and calling interface functions
0.91	March 12, 2024	Added driver download links for Guomin Download software
0.92	March 22, 2024	Added description of CAN default baud rate and function of second encoder and the initial rotor position range under the specified conditions.
1.0	2024.3.26	Added motor temperature protection instructions
1.1	May 20, 2024	Added Getting Started Guide for Calibration and Testing
1.2	May 24, 2024	Fixed CAN MIT protocol error
1.3	July 2, 2024	Added error code table
1.4	August 18, 2024	Added instructions for modifying IDs
1.5	August 21, 2024	Added instructions for using the braking resistor
1.6	August 31, 2024	Added download links for different versions of the endpoints.json file
1.7	September 5, 2024	Added CANOpen protocol documentation
1.8	October 8, 2024	Standardized exception types and added encoder error code list
1.9	October 15, 2024	Added operating instructions for the host computer software "Motor Genie"
2.0	2024.12.6	Added Modbus RTU protocol documentation and C/C++ SDK
2.1	January 23, 2025	Added Modbus random endpoint access description
2.2	May 20, 2025	Added description for PWM control and pulse direction control

Important Notes

1. Operate strictly within the parameters specified in this manual. Failure to do so may cause irreversible damage to the product!
2. During motor operation, implement overcurrent and overvoltage protection measures for the power supply to prevent damage to the driver.
3. Inspect all components for integrity before use. Contact technical support immediately if any parts are missing or damaged.
4. The driver lacks reverse polarity protection. Before connecting the power supply, refer to Section 2.4.1 to ensure correct positive and negative polarity.
5. Do not touch exposed parts of the driver to prevent electrostatic damage!

Legal Notice

Before using this product, carefully read this manual and operate the product according to its contents. The company assumes no liability for any property damage or personal injury resulting from use contrary to this manual. As this product consists of numerous components, keep children away to prevent accidents. To extend product lifespan, avoid using it in high-temperature or high-pressure environments. This manual has been printed with the utmost effort to include all functional descriptions and usage instructions. However, due to ongoing product enhancements and design modifications, discrepancies may exist between the information herein and the actual product purchased.

The colors and appearance of the actual product may differ from those depicted in this manual. The actual product shall prevail. The company reserves the right to make necessary improvements and changes to this manual at any time to correct printing errors or provide the latest information, or to enhance programs and/or equipment, without prior notice. Such changes will be incorporated into new versions of this manual, which can be obtained by contacting technical support. All illustrations are for functional reference only; the actual product shall prevail.

After-Sales Policy

After-sales service for this product strictly adheres to the Consumer Rights Protection Law of the People's Republic of China and the Product Quality Law of the People's Republic of China. Service details are as follows:

1. Warranty Period and Coverage
 - 1) Customers who purchase this product through online channels may return it without reason within seven days from the day after delivery. When returning the product, customers must present valid proof of purchase and return the invoice. Customers must ensure the returned product retains its original quality and functionality, is in good condition, and has all trademarks and labels intact on both the product and accessories. Any promotional gifts must also be returned. Returns will not be accepted if the product shows signs of intentional damage, disassembly, missing packaging, or missing components. Users are responsible for all return shipping costs. If shipping fees remain unpaid, the actual amount incurred will be deducted from the refund. Refunds will be processed within seven days of receiving the returned item. Refunds will be issued via the original payment method. Actual processing times may vary due to bank or payment institution delays.
 - 2) Within 7 days from the day after the user signs for the delivery, if a non-human-induced performance failure occurs, the company's after-sales service center will process the return after inspection and confirmation. The user must present valid proof of purchase and return the invoice. Any promotional gifts must also be returned.
 - 3) For non-human-induced performance failures occurring between 7 and 15 days after the user's signature date, the company's after-sales service center will process an exchange for the entire product set upon confirmation. The warranty period for the exchanged product will restart.

- 4) Within 15 to 365 days after the day following the user's receipt of the product, if a quality defect in the product itself is confirmed by our company's after-sales service center, free repair service will be provided. The defective product to be replaced becomes the property of our company. Products without defects will be returned in their original condition. This product has undergone rigorous testing before release. If the issue is not due to a quality defect in the product itself, we reserve the right to refuse the user's return or exchange request.

In case of any discrepancy between this manual's after-sales policy and the store's after-sales policy, the store's policy shall prevail.

2. Non-Warranty Conditions

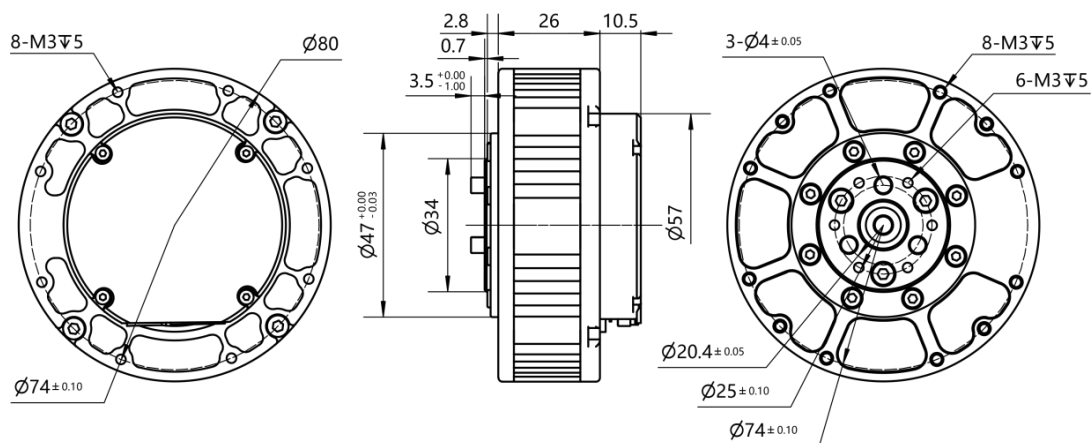
The following situations are not covered by the warranty:

- 1) Exceeding the warranty period specified in the warranty terms.
- 2) Damage or destruction caused by improper use not in accordance with the instruction manual.
- 3) Damage or destruction caused by improper operation, maintenance, installation, modification, testing, or other unauthorized use.
- 4) Normal mechanical wear and tear not caused by quality defects.
- 5) Damage occurring under non-normal operating conditions, including but not limited to drops, impacts, liquid immersion, or severe collisions.
- 6) Damage caused by natural disasters (such as floods, fires, lightning strikes, earthquakes, etc.) or force majeure events.
- 7) Damage resulting from operation exceeding the peak torque rating.
- 8) Not genuine products manufactured by our company or for which no valid proof of purchase can be provided.
- 9) Failures or damage caused by issues related to the design, technology, manufacturing, or quality of other non-product components.
- 10) Damage caused by unauthorized disassembly of this product.

If any of the above situations occur, the user shall bear the costs themselves.

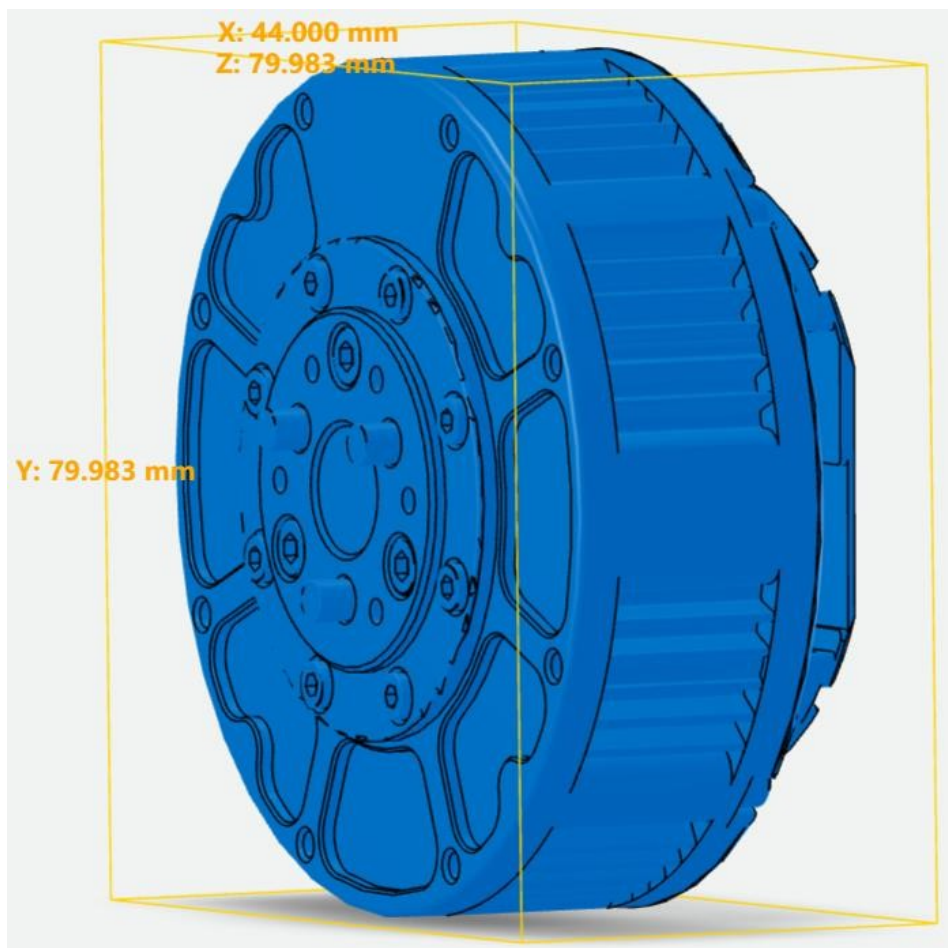
1 Motor Specifications

1.1 Drawings and Dimensions



未注公差分段						
<=6	>6<=30	>30<=120	>120<=400	>400<=2000	比例	1 : 1
± 0.1	± 0.2	± 0.3	± 0.5	± 1.0	重量kg	无

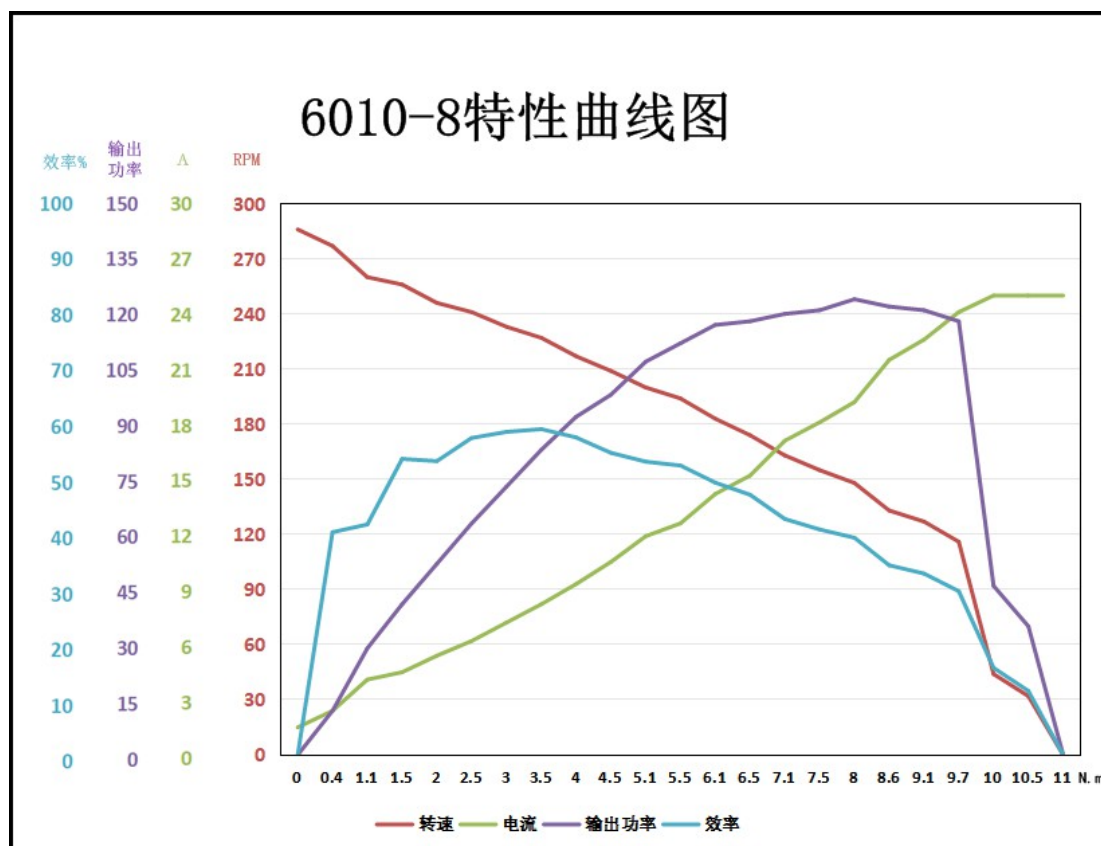
6010-8安装尺寸图



1.2 Electrical Characteristics

Rated Speed	120 rpm $\pm$ 10%
Maximum Speed	420 rpm $\pm$ 10%
Rated Torque	5 N·m
Locked-rotor torque	11 N·m
Rated Current	10.5A
Locked-rotor current	25A
No-load current	0.4A
Insulation Resistance/Stator Winding	DC 500VAC, 100M Ohms
Withstand Voltage/Stator to Frame	600 VAC, 1s, 2mA
Motor Back EMF	0.054~0.057 Vrms/rpm
Phase-to-Phase Resistance	0.48 Ω $\pm$ 10%
Phase-to-phase inductance	368 μ H $\pm$ 10%
Speed constant	12.3 rpm/V
Torque constant	0.47 N·m/A

Characteristic curve as shown below:



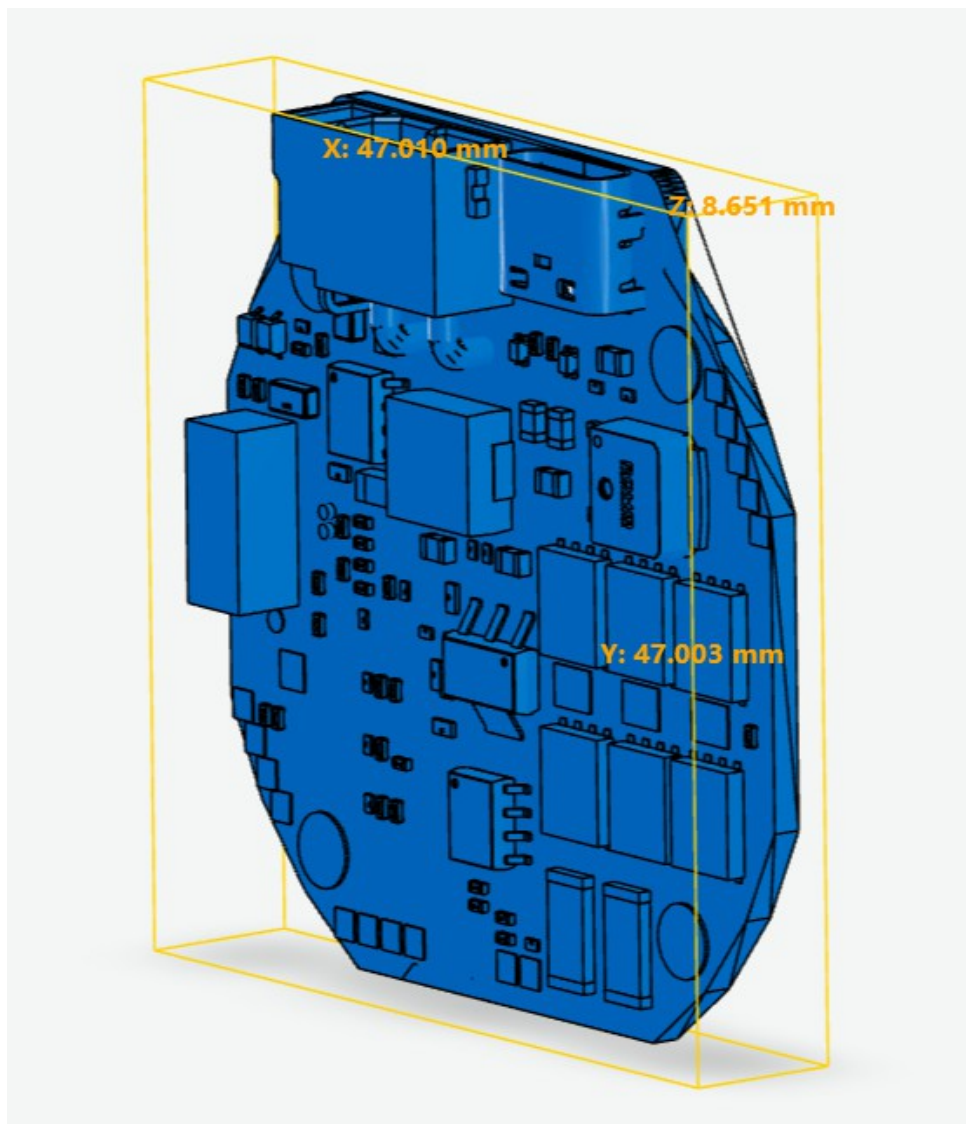
1.3 Mechanical Characteristics

Weight	388g $\pm$ 3
Number of pole pairs	14 pairs

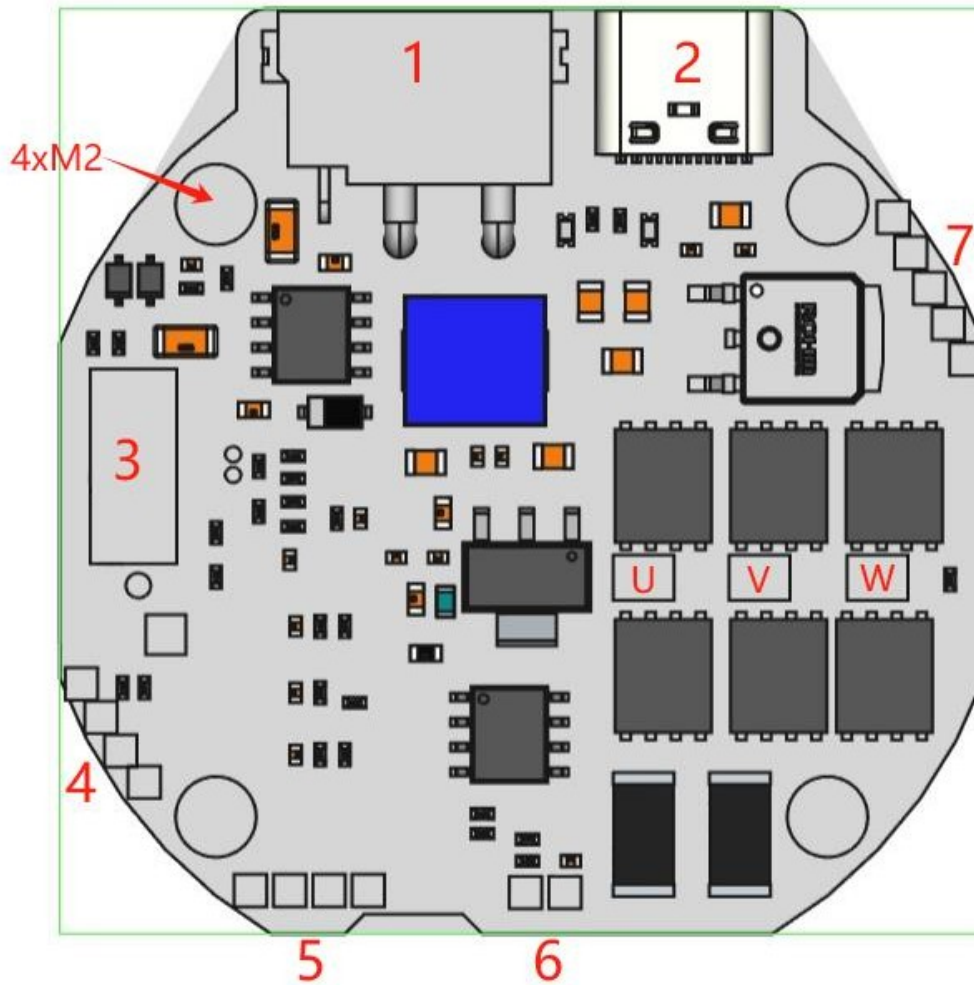
Number of Phases	3-phase
Drive Method	FOC
Reduction Ratio	8:1

2 Driver Information

2.1 Appearance and 3D Dimensions



2.2 Interface Overview



Interface Serial Number	Definition
1	15~60V Power Supply and CAN Communication Integrated Terminal
2	Type-C debugging interface and host computer communication interface
3	Interface expansion slot (supports RS485, EtherCAT, RC model, pulse direction, oil valve control interfaces/protocols)
4	SWD Debugging and Download Interface
5	Second Encoder Interface (supports I2C and UART)
6	Motor Temperature Interface (NTC)
7	Brake Resistor Interface, 12V Power Supply, Minimum/Maximum Limit Switch Interface
U/V/W	Three-phase winding welding holes
4xM2	Mounting holes

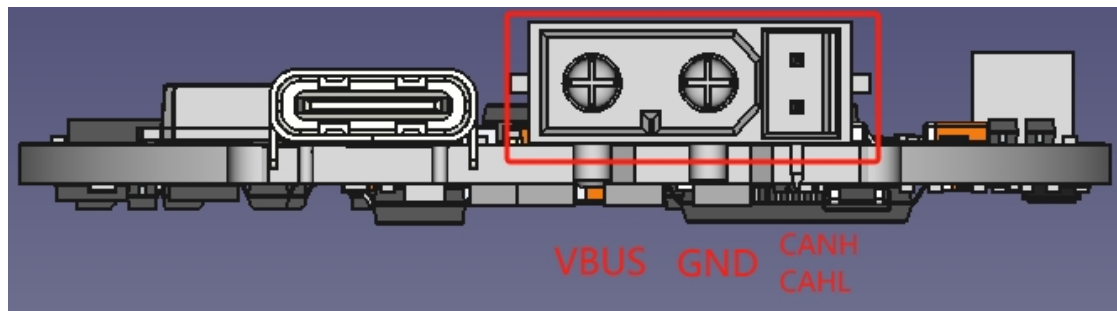
2.3 Specifications

Rated Voltage	15~48V DC
---------------	-----------

Minimum/Maximum Voltage	12/72V DC
Rated Current	6A
Maximum Line Current	30A
Maximum Phase Current	120A
Standby Power Consumption	<10mA
CAN bus maximum baud rate	1Mbps
Type-C Data Rate	10Mbps
Encoder resolution	14-bit (single-turn absolute)
Operating Temperature Range	-20°C to 70°C
Motor Temperature Alarm	90°C (adjustable)
Alarm Driver Board Temperature	90°C (adjustable)

2.4 Interface Detailed Definition

2.4.1 Power Supply and CAN Communication/RS485/PWM Terminals



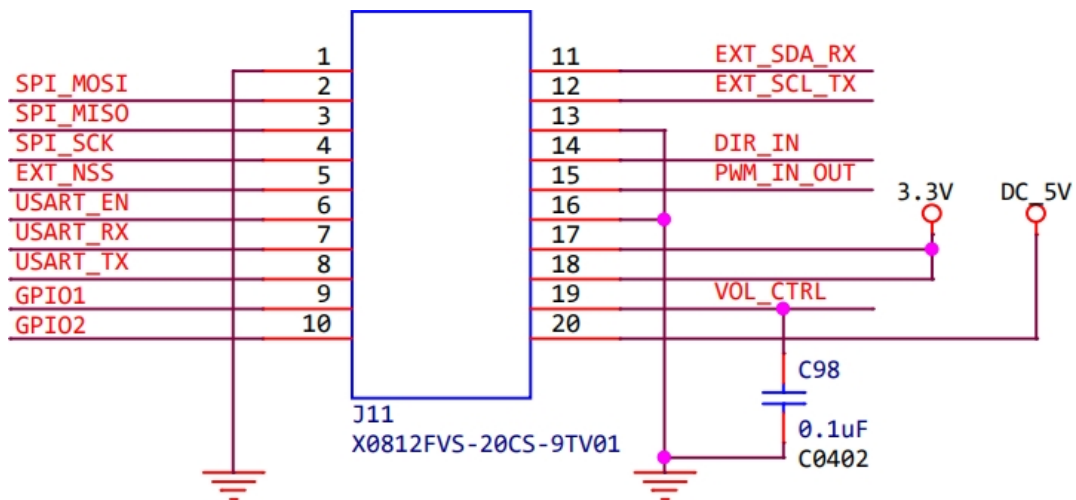
Board-mounted terminal model: XT30PB(2+2)-M; cable-end model: XT30(2+2)-F; manufacturer: AMASS. In PWM mode, CANH is the PWM signal input interface; in pulse mode, CANH is the pulse input signal interface, and CANL is the pulse direction input interface. Note that on RS485-enabled versions, CANH/CANL also serve as RS485 A/B.

2.4.2 Type-C Debug Interface

The Type-C interface uses standard data cable specifications and is compatible with common PC or mobile phone Type-C data cables.

2.4.3 Interface Expansion Slot

This slot employs the following design approach, providing rich board-to-board expansion interfaces that allow third parties to develop arbitrary expansion boards. **Note that hardware version 3.9 and later no longer provide this expansion interface.**



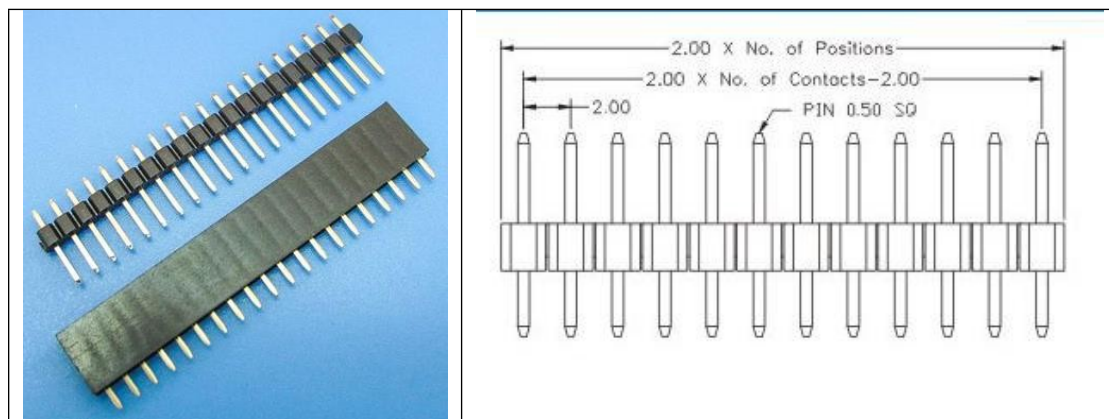
Third parties can interact with the driver via SPI, USART, I2C, PWM, ADC, GPIO, and other methods to implement various expansion functions.

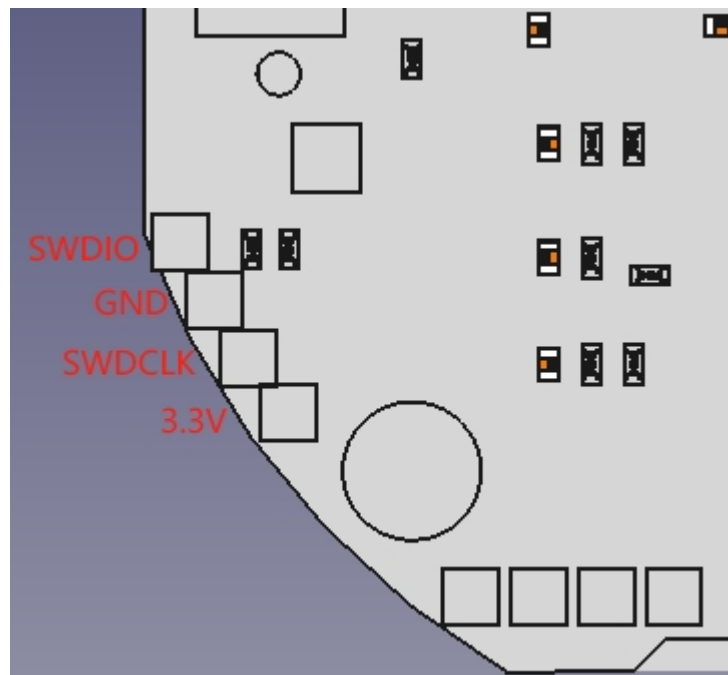
The onboard slot model is X0812FVS-20CS-9TV01 (female connector), while the expansion board slot model is X0812WVS-20AS-9TV01 (male connector), manufactured by Xingkun.

Female connector X0812FVS-20CS-9TV01	Male Connector X0812WVS-20AS-9TV01

2.4.4 SWD Debug Interface

2mm pitch pin holes allow users to solder 2mm straight single-row pins, as shown below:

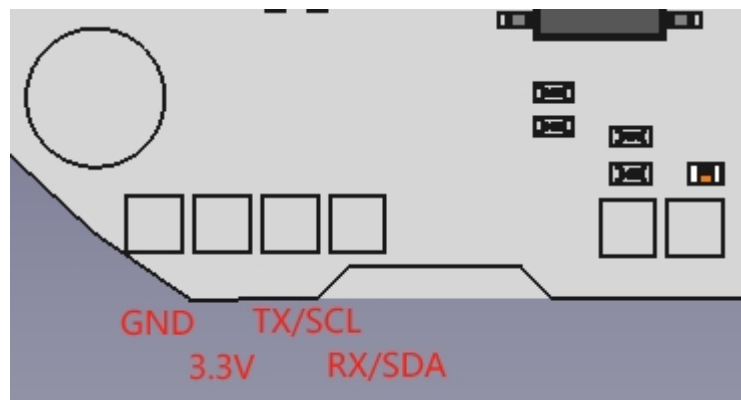




2.4.5 Second Encoder Interface

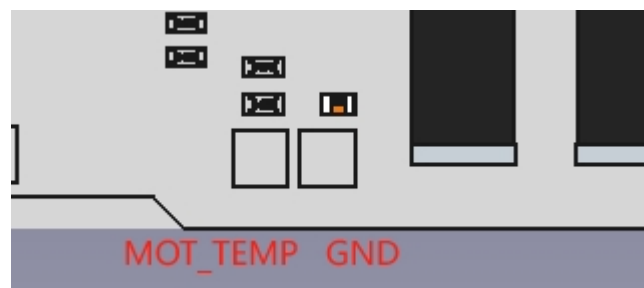
2mm pitch pin holes allow users to solder 2mm straight single-row pins. Refer to Section 2.4.4.

This interface can communicate with the second encoder via USART (TX/RX) or I2C (SCL/SDA).



2.4.6 Motor Temperature Interface

The motor incorporates a 10K NTC thermistor with two leads soldered to MOT_TEMP and GND. There is no specific wire sequence.

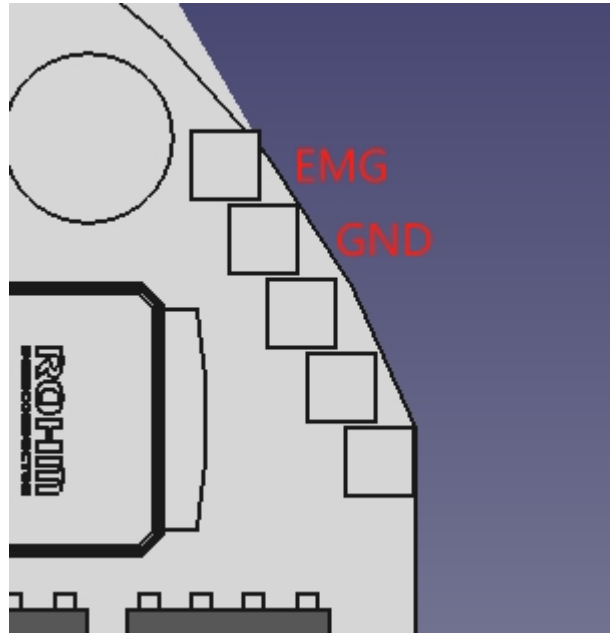


2.4.7 Brake Resistor Interface

The upper two solder pads in the 5-pin interface shown are the brake resistor interface. These are 2mm pitch pin holes, allowing users to solder a 2mm straight single-row header. See Section 2.4.4.

When configured as the brake interface, the driver continuously supplies a current to this interface upon power-up to keep the brake open, enabling normal motor operation. If the driver loses power, this current ceases, locking the brake and immobilizing the motor at the power-off position.

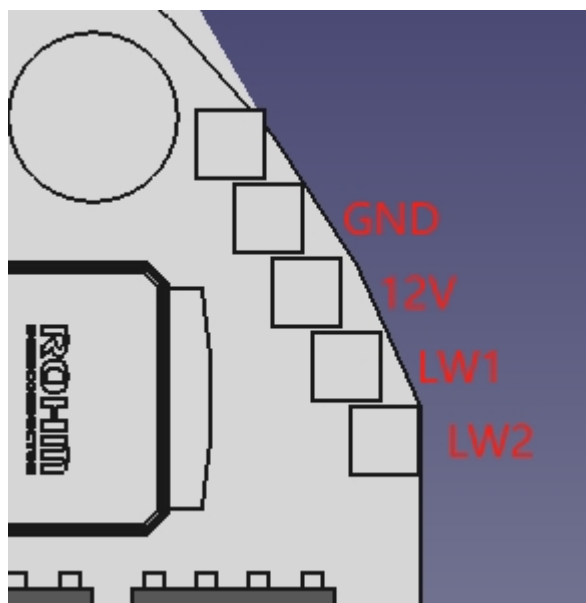
When configured as a brake resistor interface, an external brake resistor (also known as a discharge resistor) can be connected. When back EMF exceeds the threshold voltage, current is discharged through this resistor to prevent failure to emergency brake or back EMF damage to the driver.



2.4.8 Limit Switch Interface

The driver provides two limit switch interfaces and supplies 12V power to external limit switches. The pin holes are spaced 2mm apart, allowing users to solder 2mm straight single-row pins. Refer to Section 2.4.4.

LW1 is the minimum position limit switch, and LW2 is the maximum position limit switch. They can be connected to external two-wire switches or three-wire NPN switches.



2.5 Key Components and Specifications

Serial No.	Component	Model/Specification	Quantity
1	MCU	N32G455	1
2	Driver Chip	FD6288Q	1
3	Magnetic Encoder Chip	MA732, 14-bit Absolute	1
4	MOSFET	JMSH1004NG, 100V/120A	6

3 Testing Instructions

3.1 Getting Started Guide

3.1.1 Preparation

To get the motor running, you need:

- ✓ Power Supply

Refer to Chapter 1 for power supply voltage requirements. A regulated power supply or battery is recommended. A common question users often have is: How should I choose a power supply? Below are some simple suggestions for reference:

Key Considerations for Power Supply Selection:

◆ Current Requirements

Generally, it should be at least greater than 5A. The specific value depends on the system's power requirements and voltage.

◆ Voltage Requirements

Voltage requirements depend on two factors: the motor's Kv value and the system's maximum required rotational speed (RPM_{max}). The maximum required power supply voltage can be calculated using the formula:

$$V_{max} = \frac{RPM_{max}}{K_v} \times 1.25$$

where 1.25 is an empirical coefficient providing a safe voltage threshold for the system.

◆ Power Requirements

For power, simply put, it depends on the maximum current value I_{max} at the highest speed. Refer to the formula:

$$P_{max} = I_{max} \times \frac{RPM_{max}}{K_v} \times 1.25$$

- ✓ Power + Communication Interface Cable

The 6010-8 model features a 2+2 power/communication connector. Contact after-sales support for recommended 2+2 cables.

Refer to Section 2.4.1 and **ensure correct polarity connection for the power supply terminals**. **the positive and negative terminals of the power supply**. Failure to do so may result in damage to the driver, as it lacks reverse polarity protection. Additionally, **note the defined sequence of the two communication lines** (e.g., CANH/CANL). Incorrect wiring will cause communication malfunctions.

Warning

- ✧ Avoid touching the communication bus with your hands to prevent static electricity from damaging the driver's interface pins, especially in dry areas or during dry seasons!
- ✧ Never disconnect the power terminals while the device is energized!
- ✧ Avoid using circuit breakers as power switches, as this risks damaging the power pins on the driver!
- ✧ Do not exceed 72V power supply voltage!

- ✓ Type-C Data Cable

During early testing phases, it is strongly recommended to use a USB Type-C data cable for motor testing. The most common smartphone Type-C data cable is suitable; do not use Type-C cables designed solely for charging.

Note that Type-C data cables cannot power the driver, nor can they drive the motor to rotate!

✓ Powering On

First turn off the power, connect the power cable, then turn on the power. Never plug or unplug while energized, or use a circuit breaker to control only the positive or negative cable for switching. This will cause excessive inrush current and burn out the driver.

After powering on, the USB Type-C data cable can be plugged in or unplugged at any time.

3.1.2 Starting with the Host Computer



1. Install Motor Wizard

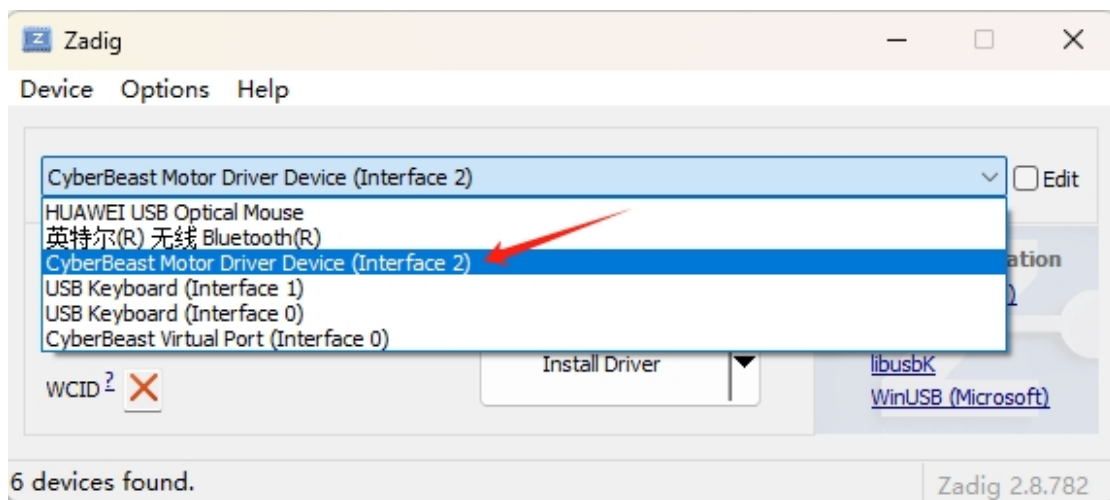
Please download the Windows installation file.

(https://bl.cyberbeast.cn/actuator/steadywin_motorwizard.exe), follow the default installation process as outlined in the steps.

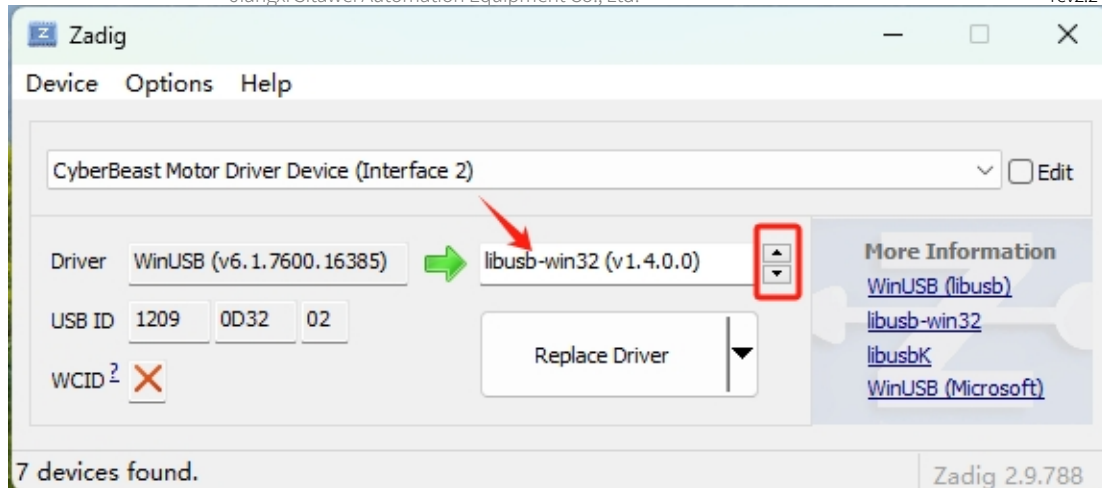
2. Installing USB Drivers

Visit <https://zadig.akeo.ie> and download the USB driver tool Zadig. Connect the driver to your computer using a Type-C cable. The driver's power indicator will illuminate. Launch Zadig, select "CyberBeast Motor Driver Device

(Interface 2)" from the dropdown menu:



Use the arrow keys to select different USB drivers. Choose the "libusb" driver version for this interface, then click "Install Driver" or "Replace Driver" to install it:



The driver is compatible with odrivetool (<https://github.com/odriverobotics/odrive.git>), so use odrivetool as the host software for debugging.

Follow these steps to install odrivetool:

➤ Windows

1. Install Python

Visit the official Python website at <https://www.python.org> to download the latest Python installer and follow the prompts to install. Do not download Python versions from third-party websites or the Microsoft Store.

2. Install Visual C++ Runtime

Install the Visual C++ Runtime Library at <https://visualstudio.microsoft.com/visual-cpp-build-tools/>. During installation, select "Desktop development with C++" as shown in the image below.



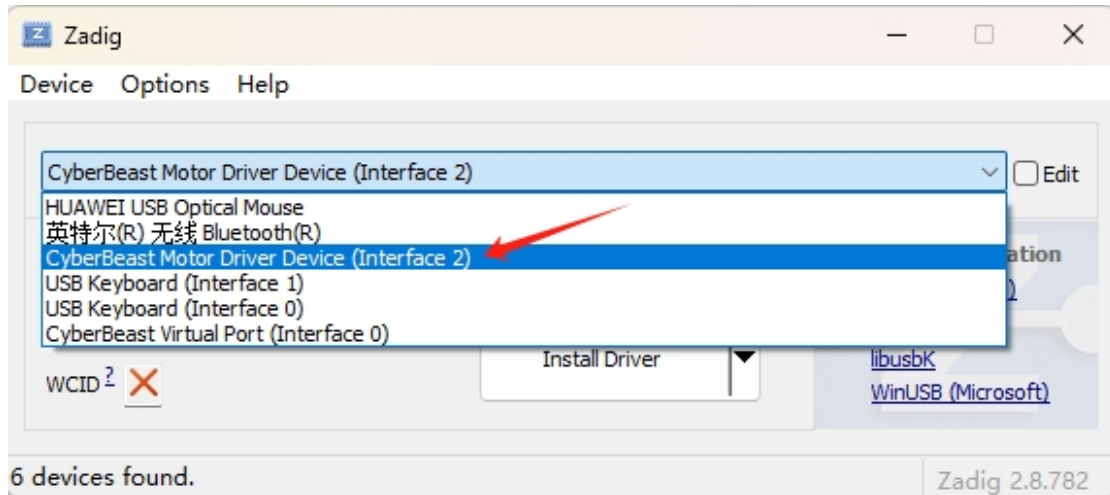
3. Install ODriveTool

Run Windows PowerShell as administrator. Execute `pip install odrive` and press Enter to install. If an error occurs during installation, please retry. If the error persists after multiple attempts, restart your computer and try again.

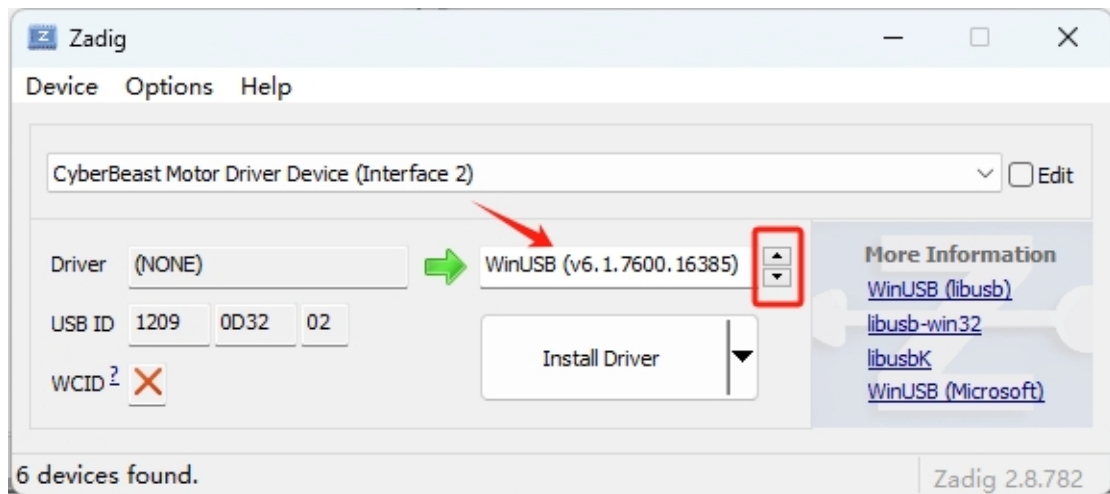
4. Install USB Driver

Visit <https://zadig.akeo.ie> and download the USB driver tool Zadig. Connect the driver to your computer using a Type-C cable. The driver's power indicator should illuminate. Open Zadig, select "CyberBeast Motor Driver Device" (Interface 2) from the dropdown menu:

(Interface 2)" from the dropdown menu:



Use the arrow keys to select different USB drivers. Choose the "WinUSB" driver version for this interface and click "Install Driver" to install it:



➤ WSL (Windows Subsystem for Linux)

1) Install python/usb/odrive tool

If you have WSL2 installed, open the WSL2 command line and follow these steps (assuming Ubuntu is installed via WSL):

```
sudo apt install python3 python3-pip sudo apt install libusb-1.0-0
sudo pip install odrive numpy matplotlib
```

The first command installs Python, the second installs USB drivers, and the third installs the ODrive tool for host computer control.

2) Connecting the drive to WSL

Plug the Type-C cable into Windows. By default, Windows will load the driver for this USB port, but WSL will not. To load this USB port into WSL, refer to the Microsoft documentation

(<https://learn.microsoft.com/zh-cn/windows/wsl/connect-usb>).

➤ Ubuntu

The installation process under Ubuntu is very similar to that under WSL. Please refer to the previous section.

3.1.3 Properly configure motor parameters

Warning

It is strongly recommended to read this section carefully, as it is crucial for successful operation and prevents motor burnout!

After successfully installing the host software (Motor Genie or odrivetool) as described in the previous section, power on the motor and connect the USB Type-C cable:

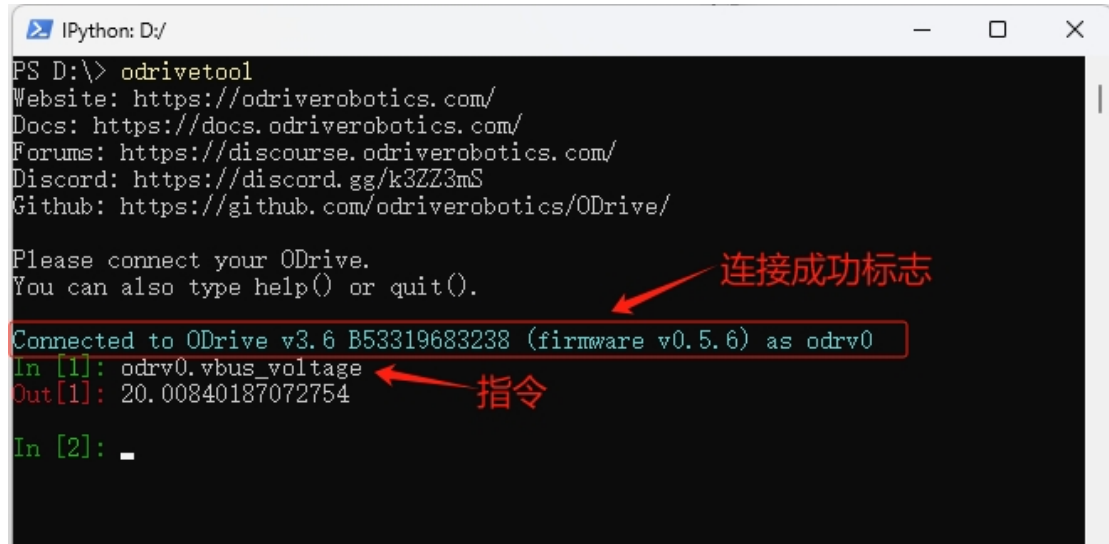


Open Motor Wizard. If the connection is successful, as shown in the left image above, the motor status and voltage will be displayed. If the connection is unsuccessful, normal voltage information cannot be displayed.

Click "Motor Parameters." The top of the interface will display the driver's hardware version and firmware version, as shown on the right in the figure above. The hardware version is 3.8.3, and the firmware version is 0.5.13.



Run odrivetool in the shell (Windows PowerShell or Linux Terminal) by typing `odrivetool` and pressing Enter. The image below shows a successful connection in Windows (connection details displayed in green text):



```
PS D:\> odrivetool
Website: https://odriverobotics.com/
Docs: https://docs.odriverobotics.com/
Forums: https://discourse.odriverobotics.com/
Discord: https://discord.gg/k3ZZ3mS
Github: https://github.com/odriverobotics/ODrive/

Please connect your ODrive.
You can also type help() or quit().

Connected to ODrive v3.6 B53319683238 (firmware v0.5.6) as odrv0
In [1]: odrv0.vbus_voltage
Out[1]: 20.00840187072754
In [2]: _
```

Command example: `odrv0.axis0.controller.input_vel`. Here, `odrv0` represents the currently connected motor (the first connected motor defaults to `odrv0`, the second to `odrv1`, and so on); `axis0` represents the first motor connected to the driver (currently only one motor connection is supported). This command queries the driver's current speed control target value.

Operation Tips

- ◆ Frequently use the TAB key to trigger command suggestions, similar to Linux command prompts. Use the arrow keys to navigate and select commands.
- ◆ The up/down keys display command history.
- ◆ While entering commands, similar commands from history will be suggested. Pressing the right arrow key will automatically complete the command.

➤ Set key thresholds (limits)



Click "Motor Parameters" in Motor Wizard (highlighted in red below) to configure key threshold parameters: voltage range, current threshold (maximum current), speed threshold (maximum speed), and calibration current.



Current limits are also influenced by temperature factors. Refer to the relevant description in the next subsection (odrivetool).

The speed limit typically refers to restrictions in speed control or position control modes, but it can also enable the speed limit switch in torque control mode. Click "Drive Parameters" in Motor Wizard to activate this switch, as shown in the red box below:



● Current Threshold

```
odrv0.axis0.motor.config.current_lim = 30
```

The above command sets the current threshold to 30A. Note that this current threshold refers to the Q-axis current, not the power supply current. This threshold value directly limits the output torque. **For the 6010-8, do not set this threshold above 50A!**

Other Factors Affecting Current Threshold

◆ Motor Temperature

If motor temperature protection is enabled (odrv0.axis0.motor.motor_thermistor.config.enabled=1), the motor's current temperature also affects the Q-axis current.

current temperature also affects the Q-axis current.

◆ Driver board temperature

If driver board temperature protection is enabled (odrv0.axis0.motor.fet_thermistor.config.enabled=1), the current temperature of the driver board also affects the Q-axis current.

The effects of both temperatures are very similar and can be expressed by the following formula:

$$I'_{lim} = \frac{T - T_l}{T_u - T_l} \times I_{lim}$$

where I is the final effective current threshold, I_{lim} is the configured current threshold, and T is the current temperature (motor temperature or drive

motor board temperature), T_l is the configured lower temperature limit (motor_thermistor.config.temp_limit_lower and fet_thermistor.config.temp_limit_lower), and T_u is the configured upper temperature limit

(motor_thermistor.config.temp_limit_upper and fet_thermistor.config.temp_limit_upper).

● Speed threshold

```
odrv0.axis0.controller.config.vel_limit = 30
```

System-wide speed limit. The above command sets it to 30 turn/s (revolutions per second). Note that by default, this speed limit does not apply in pure torque mode, but can be enabled by activating the following switch:

```
odrv0.axis0.controller.config.enable_torque_mode_vel_limit = 1
```

● Calibration Current

This current is set to 5A by default and typically requires no modification. However, if the user's power supply provides lower current, this value should be reduced to prevent low-voltage alarms during calibration.

```
odrv0.axis0.motor.config.calibration_current = 2
```

➤ Set Critical Hardware Parameters



Click "Motor Parameters" in Motor Wizard. The red box in the figure below highlights several critical hardware parameters:



For the meaning of these parameters, please refer to the description in the next subsection (odrivetool).



● Maximum Discharge/Charge Current

Discharge current refers to the forward current supplied by the power source to the driver and motor. Charge current refers to the reverse current flowing back into the power source. These values depend on the power source. Set them to appropriate levels to prevent voltage drops caused by the power source being unable to discharge or damage from back EMF. However, note that setting these values too low may easily trigger alarms.

```
odrv0.config.dc_max_negative_current odrv0.config.dc_max_positive_current
```

● Number of pole pairs

The pole number is the number of magnetic poles in the motor rotor divided by 2. Users must set this value correctly for successful calibration; otherwise, a calibration alarm will occur.

```
odrv0.axis0.motor.config.pole_pairs
```

● Torque Constant

The torque constant is the torque generated by the motor divided by the Q-axis current. Its relationship with the motor's Kv value is:

$$\text{Torque_Constant} = 8.27/K_v.$$

```
odrv0.axis0.motor.config.torque_constant
```

Whether the torque constant is correct does not affect motor operation, but it does affect unit conversion for values entered during torque control. If the user wishes to perform torque control using units of A instead of Nm, simply set this value to 1.

● Temperature Sensing

Both the driver board and motor contain NTC temperature sensors. When enabled, the driver adjusts the output current (torque) based on temperature to protect the driver and motor. 📡

```
# Motor Temperature Protection
odrv0.axis0.motor.motor_thermistor.config.enabled = 1
odrv0.axis0.motor.motor_thermistor.config.temp_limit_lower = 20
odrv0.axis0.motor.motor_thermistor.config.temp_limit_upper = 100 # Driver board temperature
protection
odrv0.axis0.motor.fet_thermistor.config.enabled = 1
odrv0.axis0.motor.fet_thermistor.config.temp_limit_lower = 20
odrv0.axis0.motor.fet_thermistor.config.temp_limit_upper = 100 # Read temperature
odrv0.axis0.motor.motor_thermistor.temperature # Motor temperature
odrv0.axis0.motor.fet_thermistor.temperature # Driver board temperature
```

➤ PID Tuning

The following process provides a reference for users to adjust PID parameters:

```
odrv0.axis0.controller.config.pos_gain=20.0 odrv0.axis0.controller.config.vel_gain=0.16
odrv0.axis0.controller.config.vel_integrator_gain=0.32
```

1. Set PID initial values
2. Set vel_integrator_gain to 0

```
odrv0.axis0.controller.config.vel_integrator_gain=0
```

3. Adjusting the vel_gain method:
 - 1) When rotating the motor in velocity control mode, if the rotation is unstable with jitter or vibration, decrease vel_gain until the rotation stabilizes.
 - 2) Then, increment vel_gain by approximately 30% each time until noticeable jitter occurs
 - 3) At this point, reduce vel_gain by about 50% to achieve stability.
4. Adjusting pos_gain:
 - 1) Rotate the motor in position mode. If rotation is unstable with jerking or vibration, decrease pos_gain until rotation stabilizes.
 - 2) Then, increment pos_gain by approximately 30% each time until position control exhibits noticeable overshoot (i.e., the motor overshoots the target position and oscillates back to it).
 - 3) Then gradually decrease pos_gain until overshoot disappears.
5. After completing these 4 adjustment steps, set vel_integrator_gain to $0.5 * \text{bandwidth} * \text{vel_gain}$, where bandwidth is the system control bandwidth. What is control bandwidth? For example, if the time from the user setting the target position to the motor actually reaching the target position is 10ms, then the control bandwidth is 100Hz. Thus, $\text{vel_integrator_gain} = 0.5 * 100 * \text{vel_gain}$.

During the above parameter tuning process, it is recommended to use the graphical tools in section 3.1.8 to monitor tuning effects in real-time, avoiding errors in visual perception. If using Motor Genius, PID tuning results are "what you see is what you get." The following figure illustrates PID parameters:



3.1.4 Power-On Calibration

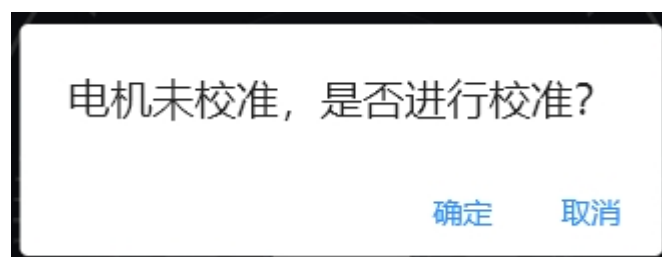
When using the micro motor for the first time, calibrate both the motor and encoder. Before calibration, secure the motor or hold it firmly by hand. Ensure the  shaft is unloaded. The calibration process is as follows:

Warning

Ensure the motor is unloaded and securely fixed before calibration!



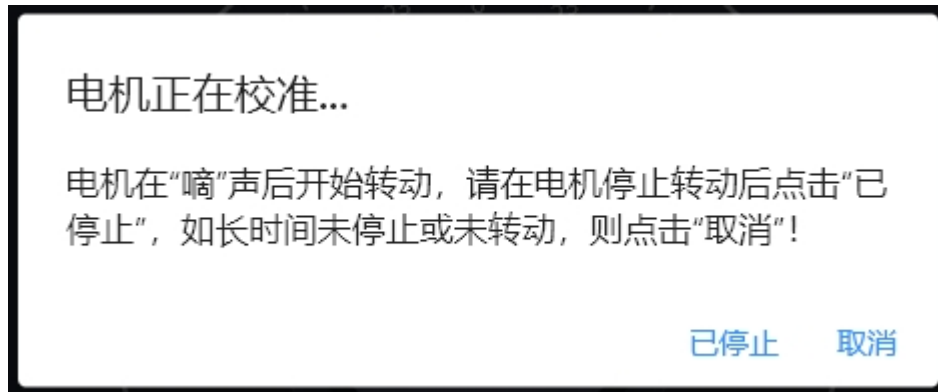
Open Motor Wizard and click "Start Motor." If the motor is uncalibrated, the following prompt will appear:



Clicking "OK" will initiate the calibration process. Alternatively, tap "Motor Parameters" on the main screen. Under "Basic Parameters,"

The motor's calibration status will be displayed as **未校准** (Not calibrated). Click the "Calibrate" button to the right of the status to enter the calibration process.

After entering the calibration process, the interface prompts:



At this point, the motor will emit a "beep" sound and begin rotating. Observe the motor's rotation. If it stops after rotating for a certain period, click "Stopped" to end the calibration process. If it does not rotate or stops after exceeding 5 minutes, click "Cancel".

After calibration completes, the driver will reboot. Please wait 1–3 seconds. You may then re-enter "Motor Parameters" to verify if it displays "已校准".



```

Python: C:\Users\yongh
PS C:\Users\yongh> odrivetool
Website: https://odriverobotics.com/
Docs: https://docs.odriverobotics.com/
Forums: https://discourse.odriverobotics.com/
Discord: https://discord.gg/k3ZZ3mS
Github: https://github.com/odriverobotics/ODrive/

Please connect your ODrive.
You can also type help() or quit().

Connected to ODrive v3.6 B948106C3537 (firmware v0.5.6) as odrv0
In [1]: odrv0.axis0.requested_state=AXIS_STATE_MOTOR_CALIBRATION

In [2]: odrv0.axis0.requested_state=AXIS_STATE_ENCODER_OFFSET_CALIBRATION

In [3]: odrv0.axis0.motor.config.pre_calibrated=1

In [4]: odrv0.axis0.encoder.config.pre_calibrated=1

In [5]: odrv0.save_configuration()
    
```

```

odrv0.axis0.requested_state=AXIS_STATE_MOTOR_CALIBRATION dump_errors(odrv0)
odrv0.axis0.requested_state=AXIS_STATE_ENCODER_OFFSET_CALIBRATION dump_errors(odrv0)

odrv0.axis0.motor.config.pre_calibrated=1
odrv0.axis0.encoder.config.pre_calibrated=1 odrv0.save_configuration()
    
```

Step-by-step explanation follows:

➤ Step 1: Motor Parameter Auto-Identification

Measure the phase resistance and phase inductance of the motor. You will hear a sharp "beep" sound. The measurement results for phase resistance and phase inductance can be viewed using the following commands:

```
odrv0.axis0.motor.config.phase_resistance odrv0.axis0.motor.config.phase_inductance
```

➤ Step 2: Check Error Codes

Check the system error codes after Step 1. If any red error codes appear, restart the motor and retry, or report to after-sales support.

➤ Step 3: Encoder Calibration

Calibrate the encoder, including aligning the encoder's installation angle with the motor's mechanical angle and calibrating the encoder itself. During this process, the motor will slowly rotate forward by one angle and then reverse by one angle. If it stops after only rotating forward, an error exists. Refer to Step 4 to check the error code.

➤ Step 4: Check Error Code

After completing the encoder calibration in Step 3, check the system error codes. The most common error encountered is ERROR_CPR_POLEPAIRS_MISMATCH, indicating an incorrect CPR setting for the encoder or an incorrect pole pair count setting for the motor. Use the following commands to view/configure these settings:

```
odrv0.axis0.encoder.config.cpr odrv0.axis0.motor.config.pole_pairs
```

➤ Step 5: Write the motor calibration success flag

➤ Step 6: Write encoder calibration success flag

➤ Step 7: Save calibration results and reboot

```
odrv0.axis0.encoder.config.pre_calibrated = 1
odrv0.axis0.motor.config.pre_calibrated = 1 odrv0.save_configuration()
```

3.1.5 Modify ID

The driver ID is unique on the bus, ranging from 0 to 63, with the default ID set to 0. If multiple motors are connected in series on the bus, each motor must be assigned a distinct ID. Users can modify the ID via USB or the bus:

➤ Modifying ID via Motor Wizard

Open Motor Wizard, connect the motor, click "Motor Parameters," modify the ID in the "Communication Parameters" section, then click "Sync" to save to the driver.



➤ Modifying ID via USB

After connecting to the driver via odrivetool, modify the ID using the following command:

```
odrv0.axis0.config.can.node_id = xxx
```

➤ Bus ID Modification

Refer to the Set_Axis_Node_ID command message in Section 4.1.2.

3.1.6 Storing and Backing Up Parameters

After modifying any parameters, ensure they are saved. Otherwise, changes will be lost after power loss or reboot. The drive will restart after saving parameters.

```
odrv0.save_configuration()
```

Parameter Backup:

```
odrivetool backup-config "d:/test.json"
```

Where "d:\test.json" is the user-defined save path and filename. The command for parameter restoration is:

```
odrivetool restore-config "d:/test.json"
```

3.1.7 Four Control Modes

After completing the preparatory steps and parameter configuration described above, you can attempt to control the motor to rotate in different modes. The 6010-8 supports position control, speed control, torque control, and motion control modes.

In Position Control mode, the following are supported: Filtered Position Control, Trajectory Control (a type of position control), and Circular Position Control.

(Trajectory Control), and Circular Position Control.

In velocity control mode, it supports direct velocity control and ramped velocity control;

In torque control mode, both direct torque control and ramped torque control are supported.

Motion control mode is a comprehensive control mode integrating position, velocity, and torque. It is typically used in scenarios requiring strong instantaneous bursts of force, such as robotic knee joints. Some industry users refer to this as MIT control mode, a term originating from the MIT open-source robotic dog, which employs this motion control mode to drive its motors.

In the subsequent detailed descriptions of each control mode, this document uses host computer and USB control commands as examples. However, the same control can be achieved using communication protocols (such as CAN), as the underlying logic remains consistent.

How to start the motor?

◆ Starting the motor (entering closed-loop control state)

For all subsequent control operations requiring motor rotation, the motor must first enter closed-loop control mode. In Motor Genius, simply click "Start Motor"; in odrivetool, the command is:

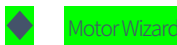
```
odrv0.axis0.requested_state = 8
```

◆ Stopping the motor (entering idle state)

Users must first place the motor in idle state when stopping operation, modifying parameters, or saving settings.

In Motor Wizard, click "Stop Motor"; in odrivetool, use the following command:

```
odrv0.axis0.requested_state = 1
```



Open Motor Wizard to see four tabs at the bottom representing four control modes: "Speed Control," "Torque Control," "Position Control," and "MIT Mode."

➤ Basic Interface Operations

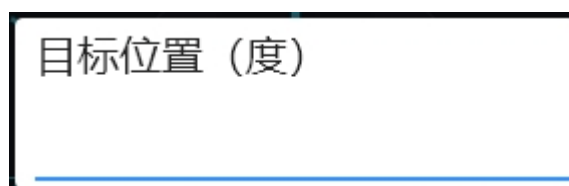
After starting the motor, the dial will glow brightly. Nearly all operations are performed on the dial:

- ✓ Clicking on the dial

A single click on the dial controls the speed/torque/position to the current dial value. Each click represents one control command. Users can click repeatedly to continuously control the motor.

- ✓ Right-click on the dial

Right-clicking on the dial opens a pop-up dialog where users can enter precise target speed/torque/position values and press Enter to execute control. Clicking outside the dialog cancels the control.



- ✓ Hold and drag on the dial

Holding and dragging the dial enables continuous control, continuously sending speed/torque/position commands to the motor with the target being the dial value at the current mouse position.

➤ **Modify Drive Parameters**

Click "Drive Parameters" to adjust settings such as control mode, speed limit, and three-loop gain (bandwidth). It is recommended to stop the motor before modifying parameters and synchronizing.



➤ **Filtered Position Control**

If you wish to generate your own position curve and send position control commands at a specific frequency, use Filtered Position Control. This mode smoothly connects and executes these commands. Using Trapezoidal Position Control in this scenario may cause the motor to exhibit jerky or granular motion.

In this mode, adjust the filter bandwidth based on the command transmission frequency. A good rule of thumb is to set the bandwidth to half the command frequency (in Hz). For example, when sending commands at 50Hz:

```
odrv0.axis0.controller.config.input_filter_bandwidth = 25
```

Enable position control filtering:

```
odrv0.axis0.controller.config.control_mode = 3
```

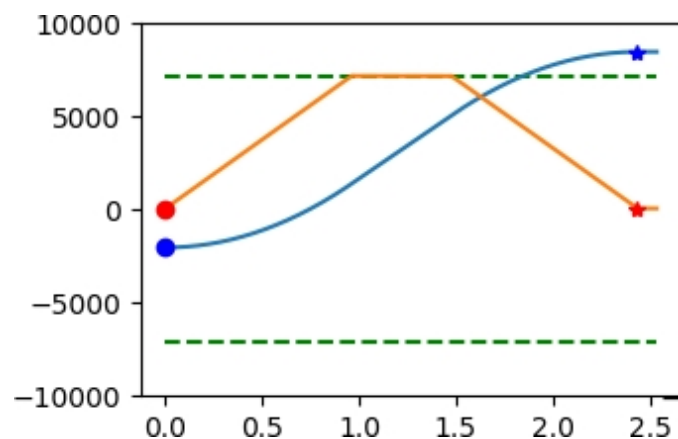
```
odrv0.axis0.controller.config.input_mode = 3
```

Then perform position control:

```
odrv0.axis0.controller.input_pos = 10 # in turns
```

➤ **Trajectory Control**

This mode allows users to set acceleration, coasting speed, and deceleration to smoothly move the motor from one position to another. The term "trapezoidal" refers to the speed curve resembling a trapezoid, as shown below where orange represents speed and blue represents position:



Adjustable control parameters:

```
odrv0.axis0.traj_traj.config.vel_limit # Maximum coasting speed, units: turn/s
odrv0.axis0.traj_traj.config.accel_limit # Maximum acceleration, units: turn/s^2
odrv0.axis0.traj_traj.config.decel_limit # Maximum deceleration, units: turn/s^2
odrv0.axis0.controller.config.inertia # Inertia, units: Nm/(turn/s^2)
```

Note that $\text{inertia} \times \text{acceleration} = \text{torque}$. This value defaults to 0. It can improve system response but is directly related to the motor load. All four values above must be greater than or equal to 0. Additionally, note that previously mentioned current and speed thresholds remain globally effective. For example, if the above coasting speed limit is set higher than the system-level `vel_limit`, the global `vel_limit` takes precedence.

To enable trapezoidal curve control mode, first:

```
odrv0.axis0.controller.config.control_mode = 3
odrv0.axis0.controller.config.input_mode = 5
```

Then perform position control:

```
odrv0.axis0.controller.input_pos = 10 # in turns
```

➤ Circular Position Control

This mode is suitable for continuous incremental position control, such as when a robot hub rotates continuously in one direction for a period or a conveyor belt runs continuously. If standard position control mode is used, the target position will gradually increase to a very large value, potentially causing positioning errors due to floating-point precision issues.

Enable:

```
odrv0.axis0.controller.config.circular_setpoints = 1
```

In this mode, each incremental step remains within a single revolution, with `input_pos` ranging from [0, 1). If `input_pos` exceeds this range, it is converted to a value within a single revolution. To enable steps exceeding one revolution, set the following parameter to a value greater than 1:

```
odrv0.axis0.controller.config.circular_setpoint_range = <N>
```

➤ Direct Velocity Control

This mode represents the simplest form of velocity control and is enabled as follows:

```
odrv0.axis0.controller.config.control_mode = 2
odrv0.axis0.controller.config.input_mode = 1
```

Then input the target speed for control:

```
odrv0.axis0.controller.input_vel = 10 # units turn/s
```

➤ **Ramped Velocity Control**

Ramped velocity control gradually increases speed to the target value at a specified slope, providing smoother acceleration than direct velocity control. Enable it as follows:

```
odrv0.axis0.controller.config.control_mode = 2
odrv0.axis0.controller.config.input_mode = 2
```

Control acceleration by adjusting the ramp

```
odrv0.axis0.controller.config.vel_ramp_rate = 0.5 # Slope unit is turn/s^2
```

rate: Then input the target velocity for control:

```
odrv0.axis0.controller.input_vel = 10 # Unit: turn/s
```

➤ **Direct Torque Control (Torque Control)**

This is the simplest torque (current) control mode. Enable it as follows:

```
odrv0.axis0.controller.config.control_mode = 1
odrv0.axis0.controller.config.input_mode = 1
```

Torque control uses Nm as its unit, while the driver firmware uses A for current. Therefore, a torque constant must be set to enable the driver to convert Nm to current, thereby driving the motor to deliver the required torque.

```
# Torque constant approximately equals 8.23/12.3Kv
odrv0.axis0.motor.config.torque_constant = 8.23/12.3
```

Then input the target speed for control:

```
odrv0.axis0.controller.input_torque = 1.2 # Unit: Nm
```

Additionally, note that if you wish to limit the maximum speed in torque mode, enable `enable\_torque\_mode\_vel\_limit` and set `vel\_limit`, e.g.:

```
odrv0.axis0.controller.config.enable_torque_mode_vel_limit = 1
odrv0.axis0.controller.config.vel_limit = 30 # units: turn/s
```

➤ **Ramped Torque Control**

Ramped Torque Control is very similar to Ramped Velocity Control. Enable it as follows:

```
odrv0.axis0.controller.config.control_mode = 1
odrv0.axis0.controller.config.input_mode = 6
```

Adjust the ramp rate as follows:


```
odrv0.axis0.controller.config.torque_ramp_rate = 0.1 # Torque ramp rate unit is Nm/s
```

➤ Motion Control (MIT Control)

Motion control mode regulates motor movement to target positions by integrating control of position, velocity, and torque. This can be expressed by the following formula:

$$T_{target} = K_p \times P_{diff} + K_d \times V_{diff} + T_{ff}$$

$$P_{diff} = P_{target} - P_{current}$$

$$V_{diff} = V_{target} - V_{current}$$

Where T_{target} is the target torque, P_{diff} is the position error, V_{diff} is the velocity error, K_p is the position control gain, K_d is the velocity control gain (or damping coefficient), and T_{ff} is the feedforward torque.

Enable motion control mode as follows:

```
odrv0.axis0.controller.config.control_mode = 3
odrv0.axis0.controller.config.input_mode = 9
```

Adjusting gains:

```
odrv0.axis0.controller.input_mit_kp = <float> # Position gain, unit Nm/turn
odrv0.axis0.controller.input_mit_kd = <float> # Damping coefficient, unit Nm/turn/s

odrv0.axis0.controller.input_vel = 30 # Units: turn/s
odrv0.axis0.controller.input_torque = 2 # Units: Nm
```

Then perform motion control by inputting input_pos, input_vel, and input_torque:

Note that during USB control, the input position, velocity, and torque all refer to the rotor side. However, when using CAN for MIT control, the position, velocity, and torque in the protocol refer to the stator side. This is to maintain consistency with the MIT open-source protocol!

3.1.8 Advanced

1) Common Command List

After successful connection, users can control the motor and retrieve operational parameters via commands. The following table lists common commands, debugging procedures, and descriptions:

Type	Command	Description
Basic Comm and	dump_errors(odrv0)	Print all error messages
	odrv0.clear_errors()	Clear all error messages
	odrv0.save_configuration()	After modifying parameters, or after the motor automatically identifies parameters or completes calibration, be sure to execute this command to save the changes. Otherwise, all changes will be lost after power-off.
	odrv0.reboot()	Reboot the driver
	odrv0.vbus_voltage	Retrieve power supply voltage (V)
	odrv0.ibus	Get power supply current (A)
	odrv0.hw_version_major	Hardware major version number. The current major version number for 6010-8 is 3
	odrv0.hw_version_minor	Hardware minor version number. The current minor version for 6010-8 is

		8
	odrv0.hw_version_variant	Different model numbers under the same hardware configuration. For 6010-8, corresponding variant number is 1
	odrv0.can.config.r120_gpio_num	Controls the GPIO number for the 120R matching resistor switch on the CAN interface odrv0.can.config.r120_gpio_num
	odrv0.can.config.enable_r120	Controls the 120R matching resistor switch for the CAN interface
	odrv0.can.config.baud_rate	CAN baud rate setting
Parameter configuration command	odrv0.config.dc_bus_undervoltage_trip_level	Low voltage alarm threshold (V)
	odrv0.config.dc_bus_overvoltage_trip_level	Overvoltage Alarm Threshold (V)
	odrv0.config.dc_max_positive_current	Maximum Line Current (Positive) (A)
	odrv0.config.dc_max_negative_current	Maximum Line Current Reverse Charging (Negative Value) (A)
	odrv0.axis0.motor.config.resistance_calibration_max_voltage	Maximum voltage value during motor parameter identification. Typically, this value is slightly less than half the power supply voltage. For example, with a 24V supply, it can be set to to 10
	odrv0.axis0.motor.config.calibration_current	Maximum current value during motor parameter identification. This value is generally set to 2~5A. It should not be too high or too low. set to 2~5A. It should not be too large or too small.
	odrv0.axis0.motor.config.torque_constant	Motor torque constant (Nm/A)
	odrv0.axis0.min_endstop.config odrv0.axis0.max_endstop.config	Minimum (LW1)/Maximum (LW2) limit switch configuration: enabled: Enable status gpio_num: Corresponding IO number. Set the minimum limit switch's IO number to 1 and the maximum limit switch's IO number to set to 2
	odrv0.axis0.encoder.config.index_offset	User-defined zero offset. This value represents the offset of the user zero point relative to the encoder zero point. After setting and saving this offset, all user-input bit will be referenced from this user zero point.
	odrv0.axis0.motor.motor_thermistor.config	Configure motor temperature sensor: enabled: Enable status temp_limit_lower: Lower temperature limit
	odrv0.axis0.motor.fet_thermistor.config	temp_limit_upper: Upper temperature limit
	odrv0.axis0.motor.motor_thermistor.temperature	Motor temperature
	odrv0.axis0.motor.fet_thermistor.temperature	Driver Temperature

Calibration Command	odrv0.axis0.requested_state=4	Perform parameter identification on the motor, including phase resistance, phase inductance, and performing phase current balance calibration.
		This process takes 3–6 seconds, during which the motor emits 🔊 a high-pitched sound. After the sound stops, or if no sound is heard after 6 seconds, run <code>dump_errors(odrv0)</code> to check for errors. Only proceed with other operations after confirming no errors exist.
	odrv0.axis0.requested_state=7	Calibrate the encoder. Before performing this operation, ensure the motor output shaft is unloaded and secure the motor with your hand or another device. After execution, the motor will rotate forward and backward for a set duration to identify and calibrate the encoder. Once the motor stops, run <code>dump_errors(odrv0)</code> to check for errors. Confirm no errors exist errors before proceeding with subsequent steps.
	odrv0.axis0.encoder.config.pre_calibrate_d=1	Writing this parameter successfully enables pre-calibration, eliminating the need for calibration at each power-up. This parameter can only be written after the above calibration succeeds ; otherwise, the write will fail.
	odrv0.axis0.controller.config.load_encoder_axis=0 er_axis=0	Ensures the current operating motor is the 0th motor. This operation is only necessary in BETA and is ineffective in the production version.
Control commands	odrv0.axis0.requested_state=1	Stop the motor and enter idle state
	odrv0.axis0.requested_state=8	Start the motor and enter closed-loop state
	odrv0.axis0.motor.config.current_limit	Maximum line current (A) during motor operation. Exceeding this value triggers an overcurrent alarm. Note that this value must not exceed 100.
	odrv0.axis0.controller.config.vel_limit	Maximum operating speed of the motor (turns per second). If the motor rotor speed exceeds this value, an overspeed alarm will be triggered. exceeding this value will trigger an overspeed alarm.
	odrv0.axis0.controller.config.enable_vel_limit	Velocity limit switch. When True, the above <code>vel_limit</code> takes effect; when False, it is disabled. is enabled; when False, it is disabled.
	odrv0.axis0.controller.config.control_mode	Control mode. 0: Voltage control 1: Torque control 2: Speed control 3: Position control

odrv0.axis0.controller.config.input_mode	Input Mode. Specifies how the user-input control value governs motor operation: 0: Idle 1: Direct Control 2: Speed ramp 3: Position filtering 5: Trapezoidal curve 6: Torque Ramp 9: Motion Control (MIT)
odrv0.axis0.controller.config.vel_gain	P value for speed loop PID control
odrv0.axis0.controller.config.vel_integrator_gain	I value for speed loop PID control
odrv0.axis0.controller.input_mit_kp	Motion Control (MIT) Position Gain
odrv0.axis0.controller.input_mit_kd	Motion Control (MIT) Speed Gain (Damping Coefficient)
odrv0.axis0.controller.config.pos_gain	P-value for Position Loop PID Control
odrv0.axis0.controller.input_torque	Torque control target, or torque feedforward (Nm) for speed control/position control Torque feedforward (Nm)
odrv0.axis0.controller.input_vel	Speed control target, or speed feedforward for position control (turn/s)
odrv0.axis0.controller.input_pos	Position control target (turns)
odrv0.axis0.encoder.set_linear_count()	Set the absolute position of the encoder. Enter 32 bit integer; the absolute value of this integer must be less than <code>odrv0.axis0.encoder.config.cpr</code>
odrv0.axis0.trap_traj.config	Contains three parameters: ➤ <code>acce_limit</code> : Maximum acceleration (rev/s ²) ➤ <code>decel_limit</code> : Maximum deceleration (rev/s ²) ➤ <code>vel_limit</code> : Maximum velocity (rev/s) These parameters take effect when <code>odrv0.axis0.controller.config.input_mode</code> is set to trapezoidal curve. Adjusting these parameters affects the acceleration and deceleration characteristics of position control. acceleration/deceleration effect.
odrv0.axis0.controller.config.input_filter_bandwidth	Position filter bandwidth. This parameter takes effect when <code>odrv0.axis0.controller.config.input_mode</code> is set to position filtering, adjusting the acceleration/deceleration effect of position control. acceleration/deceleration effect.

2) Graphical Debugging

When debugging motors, if real-time monitoring of certain operating parameters is required, leverage Python's powerful computational and graphical libraries, along with the high-throughput capability of the Type-C interface, to output motor parameters in real time.

1. Environment Preparation

Install computational and graphical libraries:

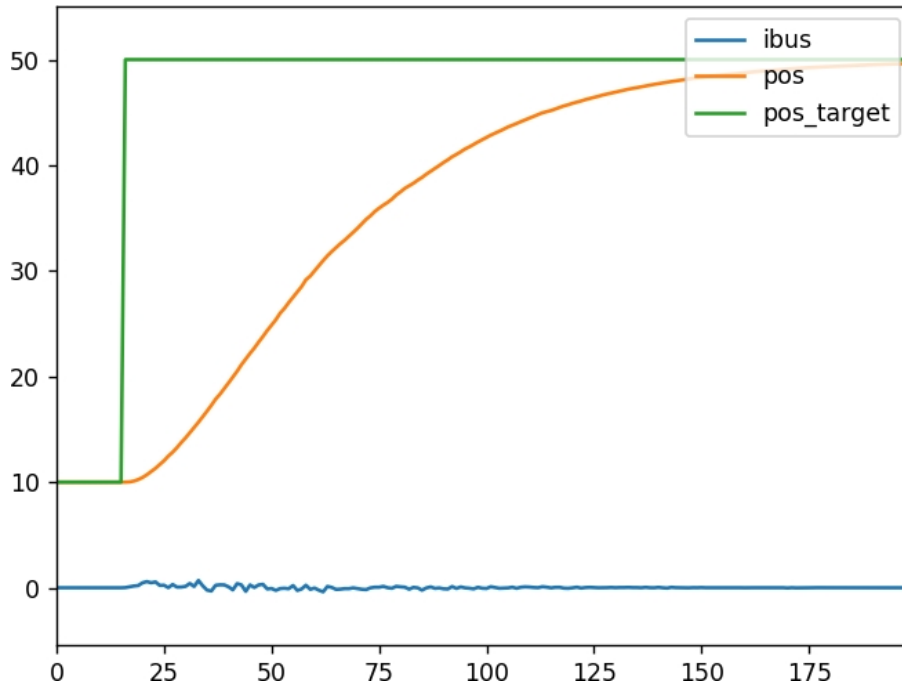
```
pip install numpy matplotlib
```

2. Visual Parameter Output

In the odrivetool command-line interface, activate the graphics library to read any motor operational metrics, such as:

```
start_liveplotter(lambda:[odrv0.ibus,odrv0.axis0.encoder.pos_estimate,  
odrv0.axis0.controller.input_pos],["ibus","pos","pos_target"])
```

This command launches a graphical interface to display the following three metrics in real time: line current, position, and target position. Subsequently, when performing position control on the motor, you will observe the real-time position control curve of the motor:



3) USB and CAN Compatibility

In earlier versions of this product (hardware version ≤ 3.7 , obtainable via the commands `odrv0.hw\_version\_major` and `odrv0.hw\_version\_minor` in Section 3.1.8 below), USB and CAN are incompatible. Switch between communication modes using the following method (ignore this section for hardware versions > 3.7):

- Switching to CAN during USB communication

When CAN is disabled, users can communicate via the Type-C interface. To switch to CAN, use the following commands:

```
odrv0.config.enable_can_a = True
odrv0.axis0.requested_state =
  AXIS_STATE_IDLE
odrv0.save_configuration()
```

- Switching to USB during CAN communication

When CAN is enabled, the user first switches the motor to idle state by sending the Set_Axis_State message (with parameter 1 indicating IDLE state), then switches to USB by sending the Disable_Can message (see Section 4.1.2).

Please note that whether switching from USB to CAN or from CAN to USB, the motor must first be in the idle (IDLE) state before switching; otherwise, the transition will fail.

4) CAN Impedance Matching Resistor Switch

The driver board includes an onboard 120-ohm impedance matching resistor. Users may enable or disable it as needed. Command examples are as follows:

```
odrv0.can.config.r120_gpio_num = 5
odrv0.can.config.enable_r120 = True
```

5) User Zero Point Configuration

By default, the position read from the motor and the input value during position control are both referenced to the zero point of the absolute encoder on the driver. However, in most user scenarios, the encoder's zero point is not the user zero point. Therefore, users need to manually set this zero point offset.

Generally, users can locate this zero point through two methods: one is via limit switches, and the other is by manually setting the zero offset—that is, the offset value of the user zero point relative to the encoder zero point:

```
# After manually rotating or using position control to reach the desired user zero position:
odrv0.axis0.encoder.config.index_offset = odrv0.axis0.encoder.pos_estimate
```

6) Limit Switches

The driver supports two limit switches (LW1 and LW2), where LW1 represents the minimum position (also the zero-return position) and LW2 represents the maximum position. To enable both limit switches, configure as follows:

```
odrv0.axis0.min_endstop.config.enabled = True odrv0.axis0.min_endstop.config.gpio_num = 1
odrv0.axis0.max_endstop.config.enabled = True odrv0.axis0.max_endstop.config.gpio_num = 2
```

When the limit switch is triggered, the system will report the MIN_ENDSTOP_PRESSED or MAX_ENDSTOP_PRESSED error, allowing the host computer to execute relevant actions at this time.

Note that hardware version 3.7 does not support limit switch functionality.

7) Braking Resistor

Users can connect a braking resistor (also known as a discharge resistor) between the braking resistor terminals (EMG and GND) described in section 2.4.7. This resistor shunts and dissipates transient currents when bus current flows back into the power supply.

Enable it via the following configuration:

```
odrv0.config.enable_brake_resistor = True odrv0.config.brake_resistance = xxx # Set the
resistance value of the external brake resistor odrv0.save_configuration()
```

How can the driver automatically discharge? Combine the following two methods to enable the controller to automatically adjust the discharge current:

- By adjusting the maximum reverse current

```
odrv0.config.max_regen_current = xxx # Maximum reverse current, positive value
```

When the reverse current exceeds the above set value, the driver will divert the excess current to the discharge resistor.

- By adjusting the back-EMF voltage range

```
odrv0.config.enable_dc_bus_overvoltage_ramp = True odrv0.config.dc_bus_overvoltage_ramp_start = xxx #  
Lower limit of shunt voltage odrv0.config.dc_bus_overvoltage_ramp_end = xxx # Upper limit of shunt  
voltage
```

When the voltage generated by back EMF exceeds the lower limit above, the driver begins shunting to the discharge resistor. When the voltage is greater than or equal to the upper limit above, the driver applies the entire voltage to the discharge resistor and attempts shunting. When the voltage is between the lower and upper limits above, the driver applies the power supply voltage (back EMF voltage) to the discharge resistor proportionally.

Note that both modes can be enabled simultaneously; the driver will attempt to combine both methods for discharge.

3.2 Firmware Update Download

Firmware can be programmed via the SWD interface (2.4.4) or Type-C interface (2.4.2), with the following three methods available:

3.2.1 National Download Software

1. USB (DFU) Programming

Note: The National Download Software supports programming via both the Type-C interface and the SWD interface (only compatible with JLink and DAP). This section primarily demonstrates programming via the Type-C interface.

First, download the National Downloader USB driver (<https://www.cyberbeast.cn/filedownload/789489>) and install the appropriate driver for your system. Then, download the National Downloader software

(<https://cyberbeast.cn/filedownload/766844>), extract it to any directory, and run it.

Next, connect the Type-C interface, enter odrivetool, and execute the following command to place the drive into DFU mode:

```
odrv0.enter_dfu_mode()
```

Finally, use the National Programmer Software for programming, as shown below. Note: After programming completes, click "Common Operations," then select "Reset" to reboot the driver. It can then be normally connected and tested via odrivetool.



2. SWD (JLink or DAP) Programming

Downloading via SWD is similar to DFU mode but requires connection through the SWD debug interface (2.4.4). Select the corresponding debug tool (JLink or DAP) as shown in the figure above.

3.2.2 pyocd

pyocd is the Python version of openOCD, supporting operations such as erase, program, and reset for universal debuggers like STLink, JLink, and DAP. **Note that the driver must be connected via the SWD interface.** For SWD pinout details, refer to

2.4.4. **The SWD interface includes a 3.3V power supply. Do not connect the wires incorrectly, as this may damage the driver!**

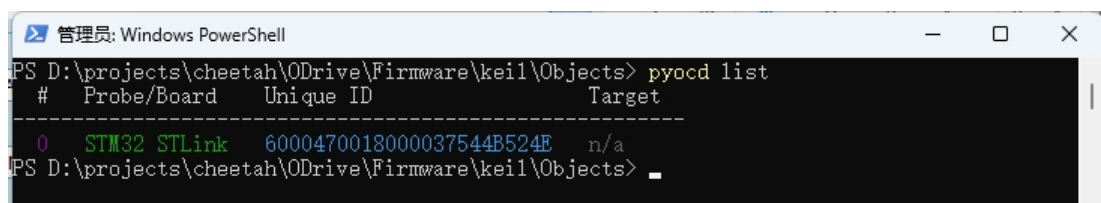
1. Installation

```
pip install pyocd
```

2. Programming

First, list the connected debugging tools:

```
pyocd list
```



Then, execute the following command to flash the bin file:

```
pyocd load .\ODrive_N32G455.bin -a 0x8000000
```

```

PS D:\projects\cheetah\ODrive\Firmware\keil\Objects> pyocd load .\ODrive_N32G455.bin -a 0x8000000
0000477 W Generic 'cortex_m' target type is selected by default; is this intentional? You will be able
to debug most devices, but not program flash. To set the target type use the '--target' argument or
target_override' option. Use 'pyocd list --targets' to see available targets types. [board]
0000529 I Loading D:\projects\cheetah\ODrive\Firmware\keil\Objects\ODrive_N32G455.bin at 0x08000000 [l
oad_cmd]
[=====] 100%
0004543 I Erased 0 bytes (0 sectors), programmed 230112 bytes (0 pages), skipped 0 bytes (0 pages) at
56.02 kB/s [loader]
PS D:\projects\cheetah\ODrive\Firmware\keil\Objects>

```

4 Communication Protocol and Examples

4.1 CAN Simple Protocol

The default communication interface is CAN, with a maximum communication rate of 1Mbps (readable and configurable via `odrv0.can.config.baud_rate`). The default baud rate is 500Kbps. **Note: In early hardware versions (≤ 3.7), USB and CAN are incompatible. Refer to Section 3.1.8, Item 3) for instructions on switching from USB to CAN.**

The default protocol for the driver is CAN Simple. If the user switches to the CANOpen protocol, they must revert to CAN Simple using the following command:

```
odrv0.can.config.protocol = 1
```

Note that on hardware versions 3.10 and above, the communication interface simultaneously supports RS485, pulse direction, and PWM. Use the following command to ensure the communication interface is CAN:

```
odrv0.config.comm_intf_mux = 0
```

In Motor Genius, you can also switch the physical interface and CAN protocol version in the communication parameters.

4.1.1 Protocol Frame Format

CAN communication uses a standard frame format: data frame with an 11-bit ID and 8 bytes of data, as shown in the table below (left: MSB, right: LSB):

Data Field	CAN ID (11 bits)		Data (8 bytes)
Segmentation	Bit 10 ~ Bit 5	Bit 4 ~ Bit 0	Byte 0 ~ Byte 7
Description	node_id	cmd_id	Communication Data

- **node_id**: Represents the unique ID of this motor on the bus. It can be read and set in `odrivetool` using `odrv0.axis0.config.can.node_id`.
- **cmd_id**: Command code indicating the message type in the protocol. See the remainder of this section for details.

- Communication Data: 8 bytes. Each parameter carried in a message is encoded as an integer or floating-point number in little-endian byte order. Floating-point numbers are encoded according to the IEEE 754 standard (encoding can be tested via <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

Taking the Set_Input_Pos message described in 4.1.2 as an example, assume its three parameters are: Input_Pos=3.14, Vel_FF=1000 (representing 1 rev/s), and Torque_FF=5000 (representing 5 Nm). Given that the Set_Input_Pos message has a CMD ID=0x00C and the driver's node ID (node_id) is set to 0x05, then:

- The 11-bit CAN ID = (0x05 << 5) + 0x0C = 0xAC
- According to the description of Set_Input_Pos in Section 4.1.2, Input_Pos occupies the first 4 bytes starting from byte 0, encoded as C3 F5 48 40 (the floating-point number 3.14 is encoded as a 32-bit IEEE 754 standard number 0x4048f5c3), Vel_FF occupies the 2 bytes starting from the 4th byte, encoded as E8 03 (1000=0x03E8), and Torque_FF occupies the 2 bytes starting from the 6th byte, encoded as 88 13 (5000=0x1388). Thus, the 8-byte communication data is:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
C3	F5	48	40	E8	03	88	13

4.1.2 Frame Messages

The following table lists all available messages:

CMD ID	Name	Direction	Parameters
0x001	Heartbeat	Motor→Host	Axis_Error Axis_State Motor_Flag Encoder_Flag Controller_Flag Traj_Done Life
0x002	Estop	Host→Motor	
0x003	Get_Error	Motor→Host	Error_Type
0x004	RxSdo	Motor→Host	
0x005	TxSdo	Motor→Host	
0x006	Set_Axis_Node_ID	Host→Motor	Axis_Node_ID
0x007	Set_Axis_State	Host→Motor	Axis_Requested_State
0x008	Mit_Control	Host→Motor	
0x009	Get_Encoder_Estimates	Motor→Host	Pos_Estimate Vel_Estimate
0x00A	Get_Encoder_Count	Motor→Host	Shadow_Count Count_In_Cpr
0x00B	Set_Controller_Mode	Host→Motor	Control_Mode Input_Mode
0x00C	Set_Input_Pos	Host→Motor	Input_Pos Vel_FF Torque_FF
0x00D	Set_Input_Vel	Host→Motor	Input_Vel Torque_FF
0x00E	Set_Input_Torque	Host→Motor	Input_Torque

0x00F	Set_Limits	Host→ Motor	Velocity_Limit Current_Limit
0x010	Start_Anticogging	Host→ Motor	
0x011	Set_Traj_Vel_Limit	Host→ Motor	Traj_Vel_Limit
0x012	Set_Traj_Accel_Limits	Host→ Motor	Traj_Accel_Limit Traj_Decel_Limit
0x013	Set_Traj_Inertia	Host→ Motor	Traj_Inertia
0x014	Get_Iq	Motor→ Host	Iq_Setpoint Iq_Measured
0x016	Reboot	Host→ Motor	
0x017	Get_Bus_Voltage_Current	Motor→ Host	Bus_Voltage Bus_Current
0x018	Clear_Errors	Host→ Motor	
0x019	Set_Linear_Count	Host→ Motor	Linear_Count
0x01A	Set_Pos_Gain	Host→ Motor	Pos_Gain
0x01B	Set_Vel_Gains	Host→ Motor	Vel_Gain Vel_Integrator_Gain
0x01C	Get_Torques	Motor→ Host	Torque_Setpoint Torque
0x01D	Get_Powers	Motor→ Host	Electrical_Power Mechanical_Power
0x01E	Disable_Can	Host→ Motor	
0x01F	Save_Configuration	Host→ Motor	

Detailed descriptions of all messages are as follows:

➤ Heartbeat

CMD ID: 0x001 (Motor→ Host)

For firmware versions less than (including) 0.5.11, the heartbeat format is as follows:

Start Byte	Name	Type	Description
0	Axis_Error	uint32	Driver error code (odrv0.axis0.error)
4	Axis_State	uint8	Driver State (odrv0.axis0.current_state)
5	Motor_Flag	uint8	1: Motor error (odrv0.axis0.motor.error is not 0) 0: Motor normal (odrv0.axis0.motor.error is 0)
6	Encoder_Flag	uint8	1: Encoder error (odrv0.axis0.encoder.error is not 0)
			0: Encoder normal (odrv0.axis0.encoder.error is 0)

7	Controller_Flag	uint8	bit7: odrv0.axis0.controller.trajecory_done, indicating whether the position curve has completed execution bit0: 1: Control error (odrv0.axis0.controller.error is not 0) 0: Control normal (odrv0.axis0.controller.error is 0)
---	-----------------	-------	--

For firmware versions greater than or equal to 0.5.12, the heartbeat format is as follows:

Start Byte	Name	Type	Description
0	Axis_Error	uint32	Driver error code (odrv0.axis0.error)
4	Axis_State	uint8	Drive Status (odrv0.axis0.current_state)
5	Flags	uint8	bit0: Motor error flag (whether odrv0.axis0.motor.error is 0) bit1: Encoder error flag (odrv0.axis0.encoder.error is not 0) bit2: Control error flag (odrv0.axis0.controller.error is not 0) bit7: odrv0.axis0.controller.trajecory_done, i.e. whether the position curve has completed execution
6	Reserved	uint8	Reserved
7	Life	uint8	Pulse message lifetime value. Increases by 1 for each heartbeat message, range 0-255. If this lifetime value is non-consecutive, it indicates heartbeat message loss , indicating unstable communication.

For firmware versions greater than or equal to 0.5.13, the heartbeat format is as follows:

Start Byte	Name	Type	Description
0	Axis_Error	uint32	Driver error code (odrv0.axis0.error)
4	Axis_State	uint8	Driver State (odrv0.axis0.current_state)
5	Flags	uint8	bit0: Motor error flag (whether odrv0.axis0.motor.error is 0) bit1: Encoder error flag (odrv0.axis0.encoder.error is not 0) bit2: Control error flag (odrv0.axis0.controller.error is not 0) bit3: System error flag (odrv0.error is not 0) bit7: odrv0.axis0.controller.trajecory_done, i.e. whether the position curve has completed execution
6	Reserved	uint8	Reserved
7	Life	uint8	Pulse message lifetime value. Increases by 1 for each heartbeat message, range 0-255. If this lifetime value is non-consecutive, it indicates heartbeat message loss , indicating unstable communication.

➤ Estop

CMD ID: 0x002 (Host→ Motor) No parameters, no data.

This command causes the motor to emergency stop and reports an ESTOP_REQUESTED exception.

➤ Get_Error

CMD ID: 0x003 (Motor→ Host)

Input (Host→ Motor):

Start Byte	Name	Type	Description
0	Error_Type	uint8	0: Motor abnormality detected 1: Encoder fault detected 3: Control fault detected 4: Detect system error

➤ Output (Motor→ Host):

Start Byte	Name	Type	Description
0	Error	uint32/uint64	Data and length returned for different Error_Type inputs: 0: Motor error (odrv0.axis0.motor.error), uint64, 8 bytes 1: Encoder error (odrv0.axis0.encoder.error), uint32, 4 bytes 3: Control error (odrv0.axis0.controller.error), uint32, 4 bytes 4: System Error (odrv0.error), uint32, 4 bytes

➤ RxSdo

CMD ID: 0x004 (Host→ Motor)

Input:

Start Byte	Name	Type	Description
0	opcode	uint8	0: Read 1: Write
1	Endpoint_ID	uint16	Please download the JSON file containing all parameter and interface function IDs: ➤ Version 0.5.13
			https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.js ➤ Version 0.5.14 https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.js on
3	Reserved	uint8	

4	Value	uint8[4]	Varies based on the Endpoint_ID; refer to the description in the JSON above. For example, if the Endpoint_ID corresponds to a read-write float value, these 4 bytes represent the IEEE-encoded float value. opcode=1 writes this value to the specified float.
---	-------	----------	---

Output  (when opcode=0 above):

Start Byte	Name	Type	Description
0	opcode	uint8	Fixed at 0
1	Endpoint_ID	uint16	Please download the JSON file containing all parameter and interface function IDs: ➤ Version 0.5.13 https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.json ➤ Version 0.5.14 https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.json
3	Reserved	uint8	
4	Value	uint8[4]	Varies based on the Endpoint_ID; refer to the description in the JSON above. For example, if the Endpoint_ID corresponds to a readable uint32 value, these 4 bytes represent a little-endian uint32.

➤ TxSdo

CMD ID: 0x005 (Motor→ Host)

Usage is identical to RxSdo when opcode=1.

➤ Set_Axis_Node_ID

CMD ID: 0x006 (Host→ Motor)

Start Byte	Name	Type	Description
0	Axis_Node_ID	uint32	odrv0.axis0.config.can.node_id Unique ID of the motor on the CAN bus

➤ Set_Axis_State

CMD ID: 0x007 (Host→ Motor)

Start Byte	Name	Type	Description
0	Axis_Requested_State	uint32	odrv0.axis0.requested_state ➤ 0: Undefined ➤ 1: Idle ➤ 3: Calibration (Motor Calibration + Encoder Calibration)) ➤ 4: Motor Calibration ➤ 7: Encoder Calibration ➤ 8: Closed-loop

➤ Mit_Control CMD
ID: 0x008

This implements the MIT open-source motion control protocol (<https://github.com/mit-biomimetics/Cheetah-Software>).

Note: When using USB control, the position, velocity, and torque inputs refer to the rotor side. However, when using CAN for MIT control, the position, velocity, and torque in the protocol refer to the stator side. This is to maintain consistency with the MIT open-source protocol!

✓ Host→ Motor

CAN Address	Meaning	Description
Frame		
Position		
BYTE0	Position: Total 16 bits, BYTE0 is the high 8 bits, BYTE1 is the low 8 bits	Actual position is double type, requiring conversion to 16-bit int type. Conversion process:
BYTE1	Outputs the multi-turn position of the axis in radians (RAD)	$\text{pos_int} = (\text{pos_double} + 12.5) * 65535 / 25$
BYTE2	Speed: Total 12 bits, with BYTE2 representing the high 8 , BYTE3[7-4] (high 4 bits) represents its low 4 bits. Represents the angular velocity of the output axis in units of RAD/s.	The actual velocity is a double-precision type and must be converted to a 12-bit int type. The conversion process is:
BYTE3		$\text{vel_int} = (\text{vel_double} + 65) * 4095 / 130$ The KP value is actually a double type and needs to be converted to a 12-bit int type. The conversion process is:
BYTE4	KP Value: Total of 12 bits, BYTE3[3-0] (low 4 bits) represent its high 4 bits, while BYTE4 represents its low 8 bits.	$\text{kp_int} = \text{kp_double} * 4095 / 500$
BYTE5	KD value: Total 12 bits. BYTE5 represents its high 8 bits, while BYTE6[7-4] (high 4 bits) represents its low 4 bits.	The KD value is actually a double type and needs to be converted to a 12-bit int type. The conversion process is:
BYTE6		$\text{kd_int} = \text{kd_double} * 4095 / 5$
BYTE7	Torque: Total 12 bits, BYTE6[3-0] (low 4 bits) are its high 4 bits, and BYTE7 is its low 8 bits. The unit is N · m.	Actual torque is a double type and needs to be converted to a 12-bit int type. The conversion process is: $\text{t_int} = (\text{t_double} + 50) * 4095 / 100$ The torque constant is measured in N · m/A.

✓ Motor→ Host

CAN Count	Meaning	Description
Data Frame		

Bit		
BYTE0	Node ID	Drive node id
BYTE1	Position: Total 16 bits, BYTE1 is high 8 bits, BYTE2 is low 8 bits Output: Multi-turn position of the axis in radians (RAD)	Actual position is double type, requiring conversion from 16-bit int type. Conversion formula: $\text{pos_double} = \text{pos_int} * 25 / 65535 - 12.5$
BYTE2		
BYTE3	Speed: Total 12 bits, BYTE3 represents the high 8 , while BYTE4[7-4] (high 4 bits) represents its low 4 bits. Represents the angular velocity of the output axis in units of RAD/s Torque: Total 12 bits, BYTE4[3-0] (low 4 bits) bits) are the high 4 bits, and BYTE5 is the low 8 bits. Unit is N·m.	Actual velocity is double type, requiring conversion from 12-bit int type. Conversion process: $\text{vel_double} = \text{vel_int} * 130 / 4095 - 65$ Actual torque is double type, requiring conversion from 12-bit int type. Conversion process: $\text{t_double} = \text{t_int} * 100 / 4095 - 50$ The torque constant is measured in N·m/A.
BYTE4		
BYTE5		

➤ Get_Encoder_Estimates CMD ID:
0x009 (Motor→ Host)

Start Byte	Name	Type	Unit	Description
0	Pos_Estimate	float32	rev	odrv0.axis0.encoder.pos_estimate Current motor rotor position
4	Vel_Estimate	float32	rev/s	odrv0.axis0.encoder.vel_estimate Current motor rotor speed

➤ Get_Encoder_Count

CMD ID: 0x00A (Motor→ Host)

Start Byte	Name	Type	Description
0	Shadow_Count	int32	odrv0.axis0.encoder.shadow_count Encoder Multi-Turn Count
4	Count_In_Cpr	int32	odrv0.axis0.encoder.count_in_cpr Encoder Single-Turn Count

➤ Set_Controller_Mode CMD ID:
0x00B (Host→ Motor)

Start Byte	Name	Type	Description
0	Control_Mode	uint32	odrv0.axis0.controller.config.control_mode Control Mode: 0: Voltage Control 1: Torque Control 2: Speed Control 3: Position control
4	Input_Mode	uint32	odrv0.axis0.controller.config.input_mode

			Input mode, indicating how user-input control values operate the motor: 0: Idle 1: Direct Control 2: Speed ramp 3: Position Filtering 5: Trapezoidal curve 6: Torque Ramp 9: Motion Control (MIT)
--	--	--	---

➤ Set_Input_Pos

CMD ID: 0x00C (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Input_Pos	float32	rev	odrv0.axis0.controller.input_pos Target value for position control.
4	Vel_FF	int16	0.001 rev/s	odrv0.axis0.controller.input_vel Velocity feedforward for position control.
6	Torque_FF	int16	0.001Nm	odrv0.axis0.controller.input_torque Torque feedforward for position control.

➤ Set_Input_Vel

CMD ID: 0x00D (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Input_Vel	float32	rev/s	odrv0.axis0.controller.input_vel Target value for velocity control.
4	Torque_FF	float32	Nm	odrv0.axis0.controller.input_torque Torque feedforward for speed control.

➤ Set_Input_Torque

CMD ID: 0x00E (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Input_Torque	float32	Nm	odrv0.axis0.controller.input_torque Target value for torque control.

➤ Set_Limits

CMD ID: 0x00F (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Velocity_Limit	float32	rev/s	odrv0.axis0.controller.config.vel_limit Global velocity limit.

4	Current_Limit	float32	A	odrv0.axis0.motor.config.current_lim Motor current limit.
---	---------------	---------	---	--

➤ Start_Anticogging

CMD ID: 0x010 (Host→ Motor)

Perform anticogging calibration.

➤ Set_Traj_Vel_Limit

CMD ID: 0x011 (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Trajectory Velocity Limit	float32	revolutions per second	odrv0.axis0.traj.config.vel_limit Velocity limit for trapezoidal position control.

➤ Set_Traj_Accel_Limits CMD ID:
0x012 (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Traj_Accel_Limit	float32	rev/s^2	odrv0.axis0.traj.config.accel_limit Acceleration limit for trapezoidal position control.
4	Traj_Decel_Limit	float32	rev/s^2	odrv0.axis0.traj.config.decel_limit Deceleration limit for trapezoidal position control.

➤ Set_Traj_Inertia

CMD ID: 0x013 (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Traj_Inertia	float32	Nm/(rev/s²)	odrv0.axis0.controller.config.inertia Inertia.

➤ Get_Iq

CMD ID: 0x014 (Motor→ Host)

Start Byte	Name	Type	Unit	Description
0	Iq_Setpoint	float32	A	odrv0.axis0.motor.current_control.Idq_setpoint Q-axis current setpoint.
4	Iq_Measured	float32	A	odrv0.axis0.motor.current_control.Iq_measured Actual measured value of Q-axis current.

➤ Reboot

CMD ID: 0x016 (Host→ Motor)

➤ Get_Bus_Voltage_Current CMD ID:
0x017 (Motor→ Host)

Start Byte	Name	Type	Unit	Description
0	Bus_Voltage	float32	V	odrv0.vbus_voltage Bus voltage.
4	Bus_Current	float32	A	odrv0.ibus Bus current.

➤ Clear_Errors

CMD ID: 0x018 (Host→ Motor)

Clear all errors and exceptions.

➤ Set_Linear_Count

CMD ID: 0x019 (Host→ Motor)

Set the absolute position of the encoder.

Start Byte	Name	Type	Description
0	Linear_Count	int32	odrv0.axis0.encoder.set_linear_count() Set the absolute position (count value) of the encoder.

➤ Set_Pos_Gain

CMD ID: 0x01A (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Pos_Gain	float32	(rev/s)/rev	odrv0.axis0.controller.config.pos_gain Position loop gain Kp.

➤ Set_Vel_Gains

CMD ID: 0x01B (Host→ Motor)

Start Byte	Name	Type	Unit	Description
0	Vel_Gain	float32	Nm/(rev/s)	odrv0.axis0.controller.config.vel_gain Velocity loop gain Kp.
4	Vel_Integrator_Gain	float32	Nm/rev	odrv0.axis0.controller.config.vel_integrator_gain Velocity Loop Gain Ki.

➤ Get_Torques

CMD ID: 0x01C (Motor→ Host)

Start Byte	Name	Type	Unit	Description
0	Torque_Setpoint	float32	Nm	odrv0.axis0.controller.torque_setpoint Torque setpoint for the current loop.
4	Torque	float32	Nm	Represents the current torque value.

➤ Get_Powers

CMD ID: 0x01D (Motor→ Host)

Start Byte	Name	Type	Description
0	Electrical_Power	float32	odrv0.axis0.controller.electrical_power Electrical Power
4	Mechanical_Power	float32	odrv0.axis0.controller.mechanical_power Mechanical Power

➤ Disable_Can

CMD ID: 0x01E (Host→ Motor)

Disable CAN and restart the driver.

➤ Save_Configuration

CMD ID: 0x01F (Host→ Motor)

Save the current configuration, apply changes, and restart.

4.1.3 CAN Protocol Practical Application

4.1.3.1 Practical Application: Power-On Calibration

The sequence for sending CAN messages is as follows:

CAN ID	Frame Type	Frame Data	Description
0x007	Data Frame	04 00 00 00 00 00 00 00	Message: Set_Axis_State Parameter: 4 Calibrate the motor
0x007	Data Frame	07 00 00 00 00 00 00 00	Message: Set_Axis_State parameter: 7 Calibrate the encoder

4.1.3.2 Practical Application: Speed Control

The sequence for sending CAN messages is as follows:

CAN ID	Frame Type	Frame Data	Description
0x00B	Data Frame	02 00 00 00 02 00 00 00	Message: Set_Controller_Mode

			Parameter: 2/2 Set controller mode to speed control. Input mode as speed ramp
0x007	Data Frame	08 00 00 00 00 00 00 00	Message: Set_Axis_State Parameter: 8 Enter closed-loop control state
0x0D	Data Frame	00 00 20 41 00 00 00 00	Message: Set_Input_Vel parameter: 10/0 Set target speed and torque feedforward, where target speed is 10 (floating point: 0x41200000), torque feedforward is 0 (floating point: 0x00000000) : 0x00000000)

4.1.3.3 Practical Application: Position Control

The sequence for sending the CAN message is as follows:

CAN ID	Frame Type	Frame Data	Description
0x00B	Data Frame	03 00 00 00 03 00 00 00	Message: Set_Controller_Mode Parameter: 3/3 Set control mode to position control. Input mode as position filtering
0x007	Data Frame	08 00 00 00 00 00 00 00	Message: Set_Axis_State Parameter: 8 Enter closed-loop control state
0x0C	Data Frame	CD CC 0C 40 00 00 00 00	Message: Set_Input_Pos Parameter: 2.2/0/0 Set target position, speed feedforward, and torque feedforward. The target position is 2.2 (floating point: 0x400CCCCD), torque feedforward and speed 0

4.1.4 CANOpen Compatibility

Interoperability with CANOpen is possible if node IDs are properly assigned. The following table lists valid node ID combinations for CANOpen and this protocol:

CANOpen node IDs	Protocol node IDs
32 ... 127	0x10, 0x18, 0x20, 0x28
64 ... 127	0x10, 0x11, 0x18, 0x19, 0x20, 0x21, 0x28, 0x29
96 ... 127	0x10, 0x11, 0x12, 0x18, 0x19, 0x1A, 0x20, 0x21, 0x22, 0x28, 0x29, 0x2A

4.1.5 Periodic Messages

Users can configure the motor to periodically send messages to the host computer without the host needing to send request messages to the motor. Periodic messages can be enabled/disabled via a series of configurations under `odrv0.axis0.config.can` (a value of 0 indicates disabled, while other values represent the periodic interval in milliseconds), as shown in the table below:

Message	odrivetool Configuration	Default
Heartbeat	<code>odrv0.axis0.config.can.heartbeat_rate_ms</code>	100
Get_Encoder_Estimates	<code>odrv0.axis0.config.can.encoder_rate_ms</code>	10
Get_Motor_Error	<code>odrv0.axis0.config.can.motor_error_rate_ms</code>	0
Get_Encoder_Error	<code>odrv0.axis0.config.can.encoder_error_rate_ms</code>	0
Get_Controller_Error	<code>odrv0.axis0.config.can.controller_error_rate_ms</code>	0
Get_Sensorless_Error	<code>odrv0.axis0.config.can.sensorless_error_rate_ms</code>	0
Get_Encoder_Count	<code>odrv0.axis0.config.can.encoder_count_rate_ms</code>	0
Get_Iq	<code>odrv0.axis0.config.can.iq_rate_ms</code>	0
Get_Sensorless_Estimates	<code>odrv0.axis0.config.can.sensorless_rate_ms</code>	0
Get_Bus_Voltage_Current	<code>odrv0.axis0.config.can.bus_vi_rate_ms</code>	0

By default, the first two periodic messages are enabled at startup, so users monitoring the CAN bus will observe these messages broadcasting at the configured intervals. Users may disable them using the following command:

```
odrv0.axis0.config.can.heartbeat_rate_ms = 0
odrv0.axis0.config.can.encoder_rate_ms = 0
```

For detailed information on each message, refer to Section 4.1.2.

4.2 CANOpen Protocol

CANOpen is a communication protocol specification for the automation field. This product implements the CiA301, CiA302, and CiA402 specifications. Before using the CANOpen protocol with this product, switch the CAN protocol to CANOpen using the following `odrivetool` command:

```
odrv0.can.config.protocol = 2
```

4.2.1 Device Description File (EDS)

The EDS file describes the device's functionality and communication parameters. To communicate with this product via CANOpen, you must import its EDS file. Download it from the following link: https://bl.cyberbeast.cn/actuator/cb_0.1.eds

4.2.2 Protocol Frame Format

CANOpen is based on standard CAN, with its 11-bit ID divided into a 4-bit function code and a 7-bit node ID (`node_id`). This allows support for up to 127 nodes on the bus (0 is reserved for broadcast). The following table illustrates the frame format:

Data Field	COB-ID / CAN ID (11 bits)		Data (8 bytes)
Segmentation	Bit10 ~ Bit7	Bit 6 to Bit 0	Byte 0 ~ Byte 7
Description	Function Code	<code>node_id</code>	Communication Data

Function code table as follows:

Communication Function	COB-ID	Description
NMT	0x000	Network Management
SYNC	0x080	Synchronization
EMCY	0x080+ <code>node_id</code>	Emergency Management
TPDO1~TPDO4	0x180/0x280/0x380/0x480+ <code>node_id</code>	Real-time Process Data Output

RPDO1~RPDO4	0x200/0x300/0x400/0x500+node_id	Real-time Process Data Receive
TSDO	0x600+node_id	Service Data Transmission
RSDO	0x580+node_id	Service Data Receive
NMT (Heartbeat)	0x700+node_id	Heartbeat

4.2.3 Frame Message

Frame message formats vary depending on the CAN ID (COB-ID).

4.2.3.1 NMT (Network Management)

NMT is used to broadcast status change commands to all nodes on the bus or to specific nodes. Its COB-ID is always 0, and the data consists of 2 bytes, as shown in the table below:

COB-ID	Data Byte 0	Data Byte 1
0x000	Status Command Word	node_id

The status command word contains:

Status Command Word	Description
0x01	Status changes to "operational"
0x02	Status changes to "stopped"
0x80	Status changed to "pre-operational"
0x81	Reset node
0x82	Reset communication

For example, after connecting a drive, the user first sends the following NMT command to activate all nodes on the bus:

COB-ID	Data
0x000	01 00

4.2.3.2 SYNC

The synchronization function synchronizes all nodes on the bus, primarily ensuring nodes synchronize PDO messages. Its COB-ID is fixed at 0x80, with an empty data field.

For example, after using the NMT command to activate nodes on the bus, users typically send a SYNC message to initiate PDO data transmission.

4.2.3.3 PDO

PDO is a communication method designed for real-time data transmission. It is used when users wish to periodically acquire node data or when users need to report data in an event-driven manner. PDO is only used when the node status is "operational," so users must first change the node status using an NMT message.

The format of the PDO message data field is determined by its mapping structure, which is not standardized but defined by the EDS file.

4.2.3.4 SDO

SDO communication enables access to the product's object dictionary data (defined by EDS). Object dictionary data can be understood as the product's parameter information. Generally, SDO is used solely for configuration processes and not for data exchange during operation. For real-time operational data exchange, PDO is recommended.

SDO is divided into two distinct processes: read and write. The following table outlines the write process (also known as upload):

	COB-ID	Data Byte0	Data Byte1	Data Byte2	Data Byte3	Data Byte4	Data Byte5	Data Byte6	Data Byte7
write	0x600+ node_id	0x23	index		subindex	data			
		0x27	index		subindex	data			/
		0x2B	index		subindex	data		/	/
		0x2F	index		subindex	data	/	/	/
response	0x580+ node_id	0x60	index		subindex	/	/	/	/
		0x80	Index		subindex	abortion code			

Read process (also called download):

	COB-ID	Data Byte0	Data Byte1	Data Byte2	Data Byte3	Data Byte4	Data Byte5	Data Byte6	Data Byte7
read	0x600+ node_id	0x40	index		subindex	/	/	/	/
response	0x580+ node_id	0x43	index		subindex	data			
		0x47	index		subindex	data			/
		0x4B	index		subindex	data		/	/
		0x4F	index		subindex	data	/	/	/
		0x80	index		subindex	abortion code			

4.2.3.5 NMT (Heartbeat)

Heartbeat messages are solely used to indicate a node's operational status to the bus, signifying that the node is functioning normally. The frame format is as follows:

COB-ID	Data Byte0
0x700+node_id	Status

4.3 RS485 Protocol (Modbus RTU)

Note that on hardware versions 3.10 and above, the communication interface simultaneously supports RS485, pulse direction, and PWM. Use the following command to ensure the communication interface is set to RS485:

```
odrv0.config.comm_intf_mux = 1
```

In Motor Genius, you can also switch the physical interface in the communication parameters.

For RS485-based Modbus RTU, the maximum communication rate is 115200 bps, with the default rate set to 115200 bps.

4.3.1 Modbus RTU

Based on the industrial standard Modbus RTU, the currently supported function codes are as follows:

Function Code	Meaning
0x04	Read single or multiple input registers
0x03	Read multiple latch registers
0x06	Write a single hold register
0x10	Write Multiple Hold Registers

4.3.2 Input Registers

Input registers are read-only and cannot be written. They represent system states or dynamic indicators such as error messages, voltage/current, etc. Currently supported input registers are as follows:

➤ 0x01: Motor Status

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 01	00 01	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	02	State0 State1	XX XX

Where State0 is always 0, and State1 is the motor status code:

Motor Status Code	Description
1	Idle
3	Fully Calibrated
4	Motor Calibration
7	Encoder Calibration
8	Closed-Loop Operation

11	Home Position
----	---------------

➤ **0x02: Error Flag**

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 02	00 01	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	02	Flag0 Flag1	XX XX

Flag0 is always 0. Flag1 is the exception flag, where each set bit 1 indicates a corresponding exception. Exceptions are defined as follows:

bit7~bit5	bit4	bit3	bit2	bit1	bit0
No meaning	Driver error	System error	Control error	Encoder Abnormalities	Motor Abnormal

➤ **0x03: Drive fault code 1**

➤ **0x04: Driver Exception Code 2**

The driver exception code is a 32-bit unsigned integer, where driver exception code 1 constitutes its upper 16 bits and driver exception code 2 constitutes its lower 16 bits. Users can only obtain the complete driver exception code by accessing register address 0x03 to retrieve both register values simultaneously. Neither register may be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 03	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 constitute the drive fault code. Refer to the fault code table in Section 5.2.

➤ **0x05: Motor fault code 1**

➤ **0x06: Motor Error Code 2**

➤ **0x07: Motor Error Code 3**

➤ **0x08: Motor Error Code 4**

Motor fault codes are 64-bit unsigned integers. Motor fault code 1 occupies bits 63 to 48, motor fault code 2 occupies bits

47 to 32, Motor Fault Code 3 covers 31 to 16, and Motor Fault Code 4 spans 15 to 0. Users can only obtain the complete motor fault code by accessing register address 0x05 to retrieve data from all four registers simultaneously. Individual registers cannot be accessed separately.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 05	00 04	XX XX

Motor Response:

Address	Function code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	8 bytes	2 bytes
XX	04	08	E0 E1 E2 E3 E4 E5 E6 E7	XX XX

E0 E1 E2 E3 E4 E5 E6 E7 form the motor fault code. Refer to the fault code table in Section 5.2.

➤ **0x09: Encoder fault code 1**

➤ **0x0A: Encoder Error Code 2**

The encoder error code is a 32-bit unsigned integer, where encoder error code 1 constitutes its upper 16 bits and encoder error code 2 constitutes its lower 16 bits. Users can only obtain the complete encoder error code by accessing register address 0x09 to retrieve both register values simultaneously. Neither register may be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 09	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 form the encoder error code. Refer to the error code table in Section 5.2.

- **0x0B: Control exception code 1**
- **0x0C: Control exception code 2**

The control exception code is a 32-bit unsigned integer, where Control Exception Code 1 constitutes its upper 16 bits and Control Exception Code 2 constitutes its lower 16 bits. Users can only obtain the complete control exception code by accessing register address 0x0B to retrieve both register values simultaneously. Neither register may be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 0B	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 constitute the control exception code. Refer to the exception code table in Section 5.2.

- **0x0D: System Error Code 1**
- **0x0E: System Error Code 2**

The system exception code is a 32-bit unsigned integer, where System Exception Code 1 constitutes its upper 16 bits and System Exception Code 2 constitutes its lower 16 bits. Users can only obtain the complete system exception code by accessing register address 0x0D to retrieve both register values simultaneously. Neither register may be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 0D	00 02	XX XX

Motor Response:

Address	Function code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 constitute the system error code. Refer to the error code table in Section 5.2.

- **0x0F: Position Feedback 1**
- **0x10: Position Feedback 2**

Position feedback is a 32-bit floating-point number, where Position Feedback 1 represents its upper 16 bits and Position

Feedback 2 represents its lower 16 bits. Users can only obtain the complete position feedback by accessing register address 0x0F to retrieve both register values simultaneously. Neither register can be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 0F	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	P0 P1 P2 P3	XX XX

P0 P1 P2 P3 constitute position feedback, where the 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

➤ **0x11: Speed Feedback 1**

➤ **0x12: Speed Feedback 2**

Speed feedback is a 32-bit floating-point number, where Speed Feedback 1 constitutes its upper 16 bits and Speed Feedback 2 constitutes its lower 16 bits. Users can only obtain the complete speed feedback by accessing register address 0x11 to retrieve both register values simultaneously. Neither register can be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 11	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	V0 V1 V2 V3	XX XX

V0 V1 V2 V3 constitute the speed feedback, where the 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

➤ **0x13: Bus Voltage Feedback 1**

➤ **0x14: Bus Voltage Feedback 2**

The bus voltage is a 32-bit floating-point number, where Bus Voltage 1 represents its upper 16 bits and Bus Voltage 2 represents its lower 16 bits. Users can only obtain the complete bus voltage by accessing register address 0x13 to retrieve both register values simultaneously. Neither register can be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes

XX	04	00 13	00 02	XX XX
----	----	-------	-------	-------

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	V0 V1 V2 V3	XX XX

V0 V1 V2 V3 form the bus voltage, where the 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

- **0x15: Bus Current Feedback 1**
- **0x16: Bus Current Feedback 2**

Bus current feedback is represented as a 32-bit floating-point number, where Bus Current Feedback 1 constitutes its upper 16 bits and Bus Current Feedback 2 constitutes its lower 16 bits.

Users can only obtain the complete bus current feedback by accessing register address 0x15 to retrieve both register values simultaneously. Neither register can be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 15	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3 form the bus current feedback, where the 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

➤ **0x17: Q-axis current target value 1**

➤ **0x18: Q-axis current target value 2**

The Q-axis current target is a 32-bit floating-point number, where Q-axis current target 1 constitutes its upper 16 bits and Q-axis current target 2 constitutes its lower 16 bits. Users can only obtain the complete Q-axis current target by accessing register address 0x17 to retrieve both register values simultaneously. Neither register can be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 17	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3 form the Q-axis current target value, where the 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

➤ **0x19: Q-axis current feedback 1**

➤ **0x1A: Q-axis current feedback 2**

The Q-axis current feedback is a 32-bit floating-point number, where Q-axis current feedback 1 represents its upper 16 bits and Q-axis current feedback 2 represents its lower 16 bits. Users can only obtain the complete Q-axis current feedback by accessing register address 0x19 to retrieve both register values simultaneously. Neither register can be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 19	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3 form the Q-axis current feedback, where the 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

- **0x31: Random endpoint access return value 1**
- **0x32: Random endpoint access return value 2**

Random endpoint access allows users to read/write any supported parameter. The term "endpoint" refers to parameters, each assigned a unique endpoint number. To read a parameter, obtain its endpoint number from the endpoint descriptor file, write it to the hold register 0xA1, then read the input register 0x31 to retrieve the parameter value. Refer to the description of hold register 0xA1 for the address to download the endpoint description file.

Random endpoint access returns a 32-bit value, which may be a 16-bit integer, a 32-bit integer, or a 32-bit floating-point number. The random endpoint access return value 1 corresponds to its upper 16 bits, while the random endpoint access return value 2 corresponds to its lower 16 bits. Users can only obtain the complete return value (parameter value) by accessing register address 0x31 and reading both register data simultaneously. Neither register can be accessed individually.

Host Request:

Address	Function Code	Register Address	Number of Registers	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	04	00 31	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	04	04	V0 V1 V2 V3	XX XX

Where V0, V1, V2, V3 represent the four bytes of the random endpoint access return value, from most significant to least significant. The 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

4.3.3 Preset Registers

Holding registers are readable and writable. Currently supported registers and their functions are as follows:

- **0x61: Emergency Stop**

Writing any value to this register will cause the motor to stop immediately.

Host Request:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 61	XX XX	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 61	XX XX	XX XX

➤ **0x62: Node ID**

Write the node ID to this node using function code 06, or read the current node ID using function code

03. Node ID write request:

Address	Function Code	Register Address	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 62	ID0 ID1	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 62	ID0 ID1	XX XX

Where ID0 and ID1 represent the upper 8 bits and lower 8 bits of the new node ID, respectively. Current node ID read request:

Address	Function code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 62	00 01	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	02	ID0 ID1	XX XX

➤ **0x63: Status Control**

Write motor status to this node using function code 06 (to change motor status).

Host Request:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 63	S0 S1	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 63	S0 S1	XX XX

Where S0 and S1 represent the upper 8 bits and lower 8 bits of the motor status, respectively. The status is defined as follows:

Motor Status Code	Description
1	Idle
3	Fully Calibrated
4	Motor Calibration
7	Encoder Calibration
8	Closed-Loop Operation
11	Home Position

For example, setting the motor status to 8 (Closed-Loop Operation) corresponds to the conventional "motor on"; setting the motor status to 1 (Idle) corresponds to the conventional "motor off".

➤ **0x64: Control Mode**

Users can modify this register's value using function code 06 to enter torque control, speed control, or position control mode, or

Use function code 03 to read the current control mode.

Control Mode Modification Request:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 64	M0 M1	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 64	M0 M1	XX XX

Read Control Mode Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 64	00 01	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	02	M0 M1	XX XX

Where M0 and M1 represent the upper 8 bits and lower 8 bits of the control mode, respectively. The control mode is defined as follows:

Control Mode	Description
1	Torque Control
2	Speed Control
3	Position Control

➤ **0x65: Input Mode**

Users can modify the value of this register using function code 06 to select different target value input modes, such as direct input, filtered input, etc. Alternatively, use function code 03 to read the current input mode.

wave input, etc., or use function code 03 to read the current input mode.

Input Mode Modification Request:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 65	M0 M1	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 65	M0 M1	XX XX

Read Input Mode Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 65	00 01	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	02	M0 M1	XX XX

Where M0 and M1 represent the high 8 bits and low 8 bits of the input mode. The input mode is defined as follows:

Control Mode	Description	Detailed Explanation
1	Direct Input	The target value entered by the user is directly used as the drive's target value
2	Speed Ramp	The drive's speed target value is achieved over a specified time period at a defined ramp rate User-input speed target value, used only for speed control
3	Position Filtering	The drive's position target value is the user-input position target value after filtering and is used solely for position control.
5	Trapezoidal Curve	The drive operates the motor along a trapezoidal velocity curve to reach the user-input target value. The rise slope (acceleration), fall slope (deceleration), and coast speed are all configurable. Refer to subsequent registers 0x70~0x75
6	Torque Ramp	The drive's torque target value will be reached over a specified time period according to a defined ramp rate the user-input target torque value over a specified time period. This applies only to torque control.

➤ **0x66: Position Control Target 1**

➤ **0x67: Position Control Target 2**

The position control target is a 32-bit floating-point number, where Position Control Target 1 constitutes its upper 16 bits and

Position Control Target 2 constitutes its lower

16 bits. Users can only obtain/modify the complete position control target by accessing register address 0x66 to simultaneously read/write both register data. Neither register can be accessed individually.

Modify/Update Position Control Target Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
---------	---------------	------------------	---------------------	-------------	----------------	-------------

1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 66	00 02	04	P0 P1 P2 P3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 66	00 02	XX XX

Read Position Control Target Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 66	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	P0 P1 P2 P3	XX XX

Where P0, P1, P2, P3 represent the four bytes of the position control target in descending order of priority. The 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

).

➤ **0x68: Speed control target 1**

➤ **0x69: Speed control target 2**

The speed control target is a 32-bit floating-point number, where Speed Control Target 1 is its upper 16 bits and Speed Control

Target 2 is its lower

. Users can only obtain/modify the complete speed control target by accessing register address 0x68 and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Speed Control Target Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 68	00 02	04	V0 V1 V2 V3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes

XX	10	00 68	00 02	XX XX
----	----	-------	-------	-------

Read Speed Control Target Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 68	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	V0 V1 V2 V3	XX XX

Where V0, V1, V2, V3 represent four bytes of speed control targets in descending order of priority. Each 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

).

- **0x6A: Torque control target 1**
- **0x6B: Torque control target 2**

The torque control target is a 32-bit floating-point number, where Torque Control Target 1 is its upper 16 bits and Torque Control

Target 2 is its lower

. Users can only obtain/modify the complete torque control target by accessing register address 0x6A and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Torque Control Target Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 6A	00 02	04	T0 T1 T2 T3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 6A	00 02	XX XX

Read Torque Control Target Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 6A	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	T0 T1 T2 T3	XX XX

Where T0, T1, T2, T3 represent the four bytes of torque control targets from high to low, encoded as 32-bit floating-point numbers according to the IEEE 754 standard (can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

).

➤ **0x6C: Speed Limit 1**

➤ **0x6D: Speed Limit 2**

The speed limit is a 32-bit floating-point number, where Speed Limit 1 is its upper 16 bits and Speed Limit 2 is its lower 16 bits. Users can only obtain/modify the complete speed limit by accessing register address 0x6C and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Speed Limit Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 6C	00 02	04	V0 V1 V2 V3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 6C	00 02	XX XX

Read Speed Limit Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 6C	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	V0 V1 V2 V3	XX XX

Where V0, V1, V2, V3 represent four bytes of velocity limits from highest to lowest, encoded as 32-bit floating-point numbers according to the IEEE 754 standard (can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

).

➤ **0x6E: Current limit 1**

➤ **0x6F: Current limit 2**

Current limit is a 32-bit floating-point number, where current limit 1 is its upper 16 bits and current limit 2 is its lower 16 bits.

Users can only obtain/modify the complete current limit by accessing register address 0x6E and simultaneously reading/writing both register data. Neither register can be accessed individually.

Current Limit Modification/Update Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 6E	00 02	04	I0 I1 I2 I3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 6E	00 02	XX XX

Read current limit request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 6E	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	I0 I1 I2 I3	XX XX

Where I0, I1, I2, and I3 represent four bytes of current limit values from high to low, encoded as 32-bit floating-point numbers according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

- **0x70: Trapezoidal Position Control Speed Limit 1**
- **0x71: Trapezoidal Position Control Speed Limit 2**

The trapezoidal position control speed limit is a 32-bit floating-point number, where Trapezoidal Position Control Speed Limit 1

constitutes its upper 16 bits, and Trapezoidal

Position Control Speed Limit 2 constitutes its lower 16 bits. Users can only obtain/modify the complete trapezoidal position control speed limit by accessing register address 0x70 to simultaneously read/write both register data. Neither register may be accessed individually.

Modify/Update Trapezoidal Position Control Speed Limit Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 70	00 02	04	V0 V1 V2 V3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 70	00 02	XX XX

Read Trapezoidal Position Control Speed Limit Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 70	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	V0 V1 V2 V3	XX XX

Where V0, V1, V2, V3 represent four bytes of trapezoidal position control speed limits from high to low. Each 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

➤ **0x72: Trapezoidal Position Control Acceleration Limit 1**

➤ **0x73: Trapezoidal Position Control Acceleration Limit 2**

The trapezoidal position control acceleration limit is a 32-bit floating-point number, where Trapezoidal Position Control

Acceleration Limit 1 constitutes its upper 16 bits,

while Trapezoidal Position Control Acceleration Limit 2 constitutes its lower 16 bits. Users can only obtain/modify the complete trapezoidal position control acceleration limit by accessing register address 0x72 and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Trapezoidal Position Control Acceleration Limit Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code

1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 72	00 02	04	A0 A1 A2 A3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 72	00 02	XX XX

Read Trapezoidal Position Control Acceleration Limit Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 72	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	A0 A1 A2 A3	XX XX

Where A0, A1, A2, A3 represent four bytes of trapezoidal position control acceleration limits from high to low. Each 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

- **0x74: Trapezoidal Position Control Deceleration Limit 1**
- **0x75: Trapezoidal Position Control Deceleration Limit 2**

The deceleration limit for trapezoidal position control is a 32-bit floating-point number, where Deceleration Limit 1 represents its upper 16 bits, while Trapezoidal Position Control Deceleration Limit 2 constitutes its lower 16 bits. Users can only obtain/modify the complete trapezoidal position control deceleration limit by accessing register address 0x74 and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Trapezoidal Position Control Deceleration Limit Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 74	00 02	04	D0 D1 D2 D3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 74	00 02	XX XX

Read Trapezoidal Position Control Deceleration Limit Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 74	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	D0 D1 D2 D3	XX XX

Where D0, D1, D2, D3 represent four bytes of trapezoidal position control acceleration limit values from high to low. Each 32-bit floating-point number is encoded according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

➤ **0x76: Inertia 1**

➤ **0x77: Inertia 2**

Inertia is represented as a 32-bit floating-point number, where Inertia 1 constitutes its upper 16 bits and Inertia 2 its lower 16 bits. Users can only obtain/modify the complete inertia value by accessing register address 0x76 to simultaneously read/write both register data. Neither register can be accessed individually.

Modify/Update Inertia Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 76	00 02	04	I0 I1 I2 I3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 76	00 02	XX XX

Read Inertia Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 76	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes

XX	03	04	I0 I1 I2 I3	XX XX
----	----	----	-------------	-------

Where I0, I1, I2, I3 represent the four bytes of inertia from highest to lowest, encoded as 32-bit floating-point numbers according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

- **0x78: Position gain 1**
- **0x79: Position Gain 2**

Position gain is a 32-bit floating-point number, where Position Gain 1 constitutes its upper 16 bits and Position Gain 2 constitutes its lower 16 bits. Users can only obtain/modify the complete position gain by accessing register address 0x78 and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Position Gain Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 78	00 02	04	G0 G1 G2 G3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 78	00 02	XX XX

Read Position Gain Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 78	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	G0 G1 G2 G3	XX XX

Where G0, G1, G2, G3 represent the four bytes of position gain from high to low, encoded as 32-bit floating-point numbers according to the IEEE 754 standard (can be tested via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

).

- **0x7A: Speed gain 1**
- **0x7B: Speed Gain 2**

The speed gain is a 32-bit floating-point number, where Speed Gain 1 is its upper 16 bits and Speed Gain 2 is its lower 16 bits. Users can only obtain/modify the complete speed gain by accessing register address 0x7A and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Speed Gain Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Checksum

1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 7A	00 02	04	G0 G1 G2 G3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 7A	00 02	XX XX

Read Speed Gain Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 7A	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	G0 G1 G2 G3	XX XX

G0, G1, G2, and G3 represent four bytes of velocity gain from highest to lowest, encoded as 32-bit floating-point numbers according to the IEEE 754 standard (verifiable via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

).

- **0x7C: Speed integral gain 1**
- **0x7D: Velocity integral gain 2**

The velocity integral gain is a 32-bit floating-point number, where velocity integral gain 1 occupies the upper 16 bits and velocity

integral gain 2 occupies the lower

. Users can only obtain/modify the complete speed integral gain by accessing register address 0x7C and simultaneously reading/writing both register data. Neither register can be accessed individually.

Modify/Update Speed Integral Gain Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 7C	00 02	04	G0 G1 G2 G3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 7C	00 02	XX XX

Read Speed Integral Gain Request:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	03	00 7C	00 02	XX XX

Motor Response:

Address	Function Code	Byte Length	Register Value	Parity Code
1 byte	1 byte	1 byte	2 bytes	2 bytes
XX	03	04	G0 G1 G2 G3	XX XX

Where G0, G1, G2, G3 represent four bytes of velocity integral gain from highest to lowest, encoded as 32-bit floating-point numbers according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

➤ 0x7E: Restart

Writing any value to this register will cause the motor driver to restart. Host

Request:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 7E	XX XX	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 7E	XX XX	XX XX

➤ **0x7F: Clear Exceptions**

Writing any value to this register clears all errors and exceptions. Host Request:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 7F	XX XX	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 7F	XX XX	XX XX

➤ **0x80: Configuration Storage**

Writing any value to this register saves all current parameters to the drive's Flash memory for permanent storage.

Host Request:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 80	XX XX	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 80	XX XX	XX XX

➤ **0xA1: Random endpoint access endpoint number**

Random endpoint access allows users to read/write any supported parameter. The term "endpoint" refers to parameters, each of which has a unique endpoint number. When a user wishes to read/write a specific parameter, they can obtain its endpoint number from the endpoint description file. Write this number to hold register 0xA1, then read input register 0x31 to retrieve the parameter value, or write to hold register 0xA2 to update the parameter value.

Endpoint numbers are 16-bit unsigned integers. Download the JSON file containing endpoint numbers for all parameters and interface functions:

- ✓ Version 0.5.13: https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.json
- ✓ Version 0.5.14: https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.json

Write to random endpoint request:

Address	Function code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 A1	E0 E1	XX XX

Motor Response:

Address	Function Code	Register Address	Register Value	Checksum
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	06	00 A1	E0 E1	XX XX

Where E0 and E1 are two bytes representing the 16-bit endpoint number in high-to-low byte order.

- **0xA2: Random endpoint access write value 1**
- **0xA3: Random endpoint access writes value 2**

The random endpoint access write value is a 32-bit number, which could be a 16-bit integer, a 32-bit integer, or a 32-bit floating-point number.

random endpoint access write value 1 represents its high 16 bits, and random endpoint access write value 2 represents its low 16 bits. Users can only update the endpoint value (parameter value) by accessing register address 0xA2 and writing data to both registers simultaneously. Neither register can be accessed individually.

Write Request:

Address	Function Code	Register Address	Number of Registers	Byte Length	Register Value	Parity Code
1 byte	1 byte	2 bytes	2 bytes	1 byte	4 bytes	2 bytes
XX	10	00 A2	00 02	04	V0 V1 V2 V3	XX XX

Motor Response:

Address	Function Code	Register Address	Number of Registers	Parity Code
1 byte	1 byte	2 bytes	2 bytes	2 bytes
XX	10	00 A2	00 02	XX XX

Where V0, V1, V2, and V3 represent the four bytes of the write value in descending order. If this endpoint is a 32-bit floating-point number, it shall be encoded according to the IEEE 754 standard (encoding can be verified via the website <https://www.h-schmidt.net/FloatConverter/IEEE754.html>).

4.3.4 Modbus Protocol in Practice

Assume the motor address is 1.

1) Practical Application: Speed Control

The sequence for sending a frame is as follows:

Frame Data	Register	Description
01 06 00 64 00 02 XX XX	Hold register 0x64	Set control mode to 2 (speed control)
01 06 00 65 00 02 XX XX	Hold register 0x65	Set input mode to 2 (speed ramp)
01 06 00 63 00 08 XX XX	Hold register 0x63	Change motor state to 8 (closed-loop operation).
01 10 00 68 00 02 04 40 13 33 33 XX XX	Hold register 0x68	Set target speed to 2.3 r/s, converted to 32 bit floating-point number: 40 13 33 33.

2) Practical Application: Position Control

The sequence for sending frames is as follows:

Frame Data	Register	Description
01 06 00 64 00 03 XX XX	Preset Register 0x64	Set control mode to 3 (position control)
01 06 00 65 00 03 XX XX	Hold register 0x65	Set input mode to 3 (position filtering)
01 06 00 63 00 08 XX XX	Hold register 0x63	Change motor status to 8 (closed-loop operation).
01 10 00 68 00 02 04 40 49 0F F9 XX XX	Hold register 0x68	Set target speed to 3.1416r, converted to a 32-bit floating-point number: 40 49 0F F9.

4.4 PWM Input Control

Please ensure the hardware version is 3.10 or later before using this feature.

This feature supports controlling various writable parameters such as position, speed, or torque via standard RC PWM signals (1ms–2ms pulse width, 50Hz). By configuring GPIO pins to PWM mode and setting mapping relationships, direct motor control from an RC receiver or other PWM controllers can be achieved.

4.4.1 Wiring Instructions

Connect the PWM signal wire to the CANH position described in Section 2.4.1. This interface supports PWM signals of 5V or lower. Ensure the PWM signal shares a common ground with the motor driver.

4.4.2 Software Configuration

- Switching the communication interface to PWM:

```
odrv0.config.comm_intf_mux = 3
```

In Motor Wizard, you can also switch the physical interface in the communication parameters. Please locate this option in the motor parameters.



- Set GPIO to PWM mode

```
odrv0.config.gpio1_mode = GPIO_MODE_PWM
```

- Set PWM mapping control mode

Can be mapped to speed/position/torque mode. Configure one of the following modes as needed. Speed Mode

Set the motor control mode to Speed Ramp mode (control_mode = 2, input_mode = 2)

```
odrv0.config.gpio1_pwm_mapping.endpoint = odrv0.axis0.controller._input_vel_property
```

Position Mode

Set the motor control mode to Trapezoidal curve mode (control_mode = 3, input_mode = 5)

```
odrv0.config.gpio1_pwm_mapping.endpoint = odrv0.axis0.controller._input_pos_property
```

Torque Mode

Set the motor control mode to Speed Ramp Mode (control_mode = 1, input_mode = 6)

```
odrv0.config.gpio1_pwm_mapping.endpoint = odrv0.axis0.controller._input_torque_property
```

➤ Set PWM mapping parameters

```
odrv0.config.gpio1_pwm_mapping.min = -8.0
odrv0.config.gpio1_pwm_mapping.max = 8.0
```

min and max define the range of control values mapped to the PWM pulse width (1ms~2ms). Internally, the driver linearly maps the pulse width value to a real number within the [min, max] range, then writes it to the endpoint control target value you configured. min and max can be set to any value as needed.

➤ Save configuration (optional)

```
odrv0.save_configuration()
```

After completing the above configuration, enter closed-loop mode to enable PWM control.

4.5 Pulse/Direction Control

Ensure hardware version is 3.10 or higher before using this feature, as it conflicts with the limit switch function. This is the simplest method to control the driver, but also the most primitive and fragile. Therefore, avoid using this control method unless absolutely necessary.

This function supports controlling brushless motors via the traditional Step/Dir (pulse/direction) interface, enabling compatibility with standard stepper motor drivers. In this mode:

Step (pulse) signal: Each rising edge corresponds to the motor moving one preset step (determined by steps_per_circular_range). Maximum recommended step frequency: 50kHz

Dir (Direction) signal: Level determines movement direction (High level: clockwise / Low level: counterclockwise)

4.5.1 Wiring Instructions

The Step (pulse signal) wire connects to the CANH position described in Section 2.4.1. The Dir (direction signal) wire connects to the CANL position described in Section 2.4.1. This interface supports pulse signals of 5V or lower. Ensure the Step and Dir signals share a common ground with the motor driver; otherwise, signal recognition may fail.

to the CANL position described in Section 2.4.1. This interface supports pulse signals at 5V or below. Ensure the Step and Dir signals share a common ground with the motor driver; otherwise, signal recognition may fail.

4.5.2 Software Configuration

➤ Switch communication interface to Step/Dir:

```
odrv0.config.comm_intf_mux = 2
```

In Motor Wizard, you can also switch the physical interface in the communication parameters. Locate this option in the motor parameters.



- Set GPIO to Step/Dir mode

```
odrv0.axis0.config.step_gpio_pin = 6
odrv0.axis0.config.dir_gpio_pin = 7
```

- Configure Step/Dir pins

```
odrv0.config.gpio6_mode = GPIO_MODE_DIGITAL_PULL_UP
odrv0.config.gpio7_mode = GPIO_MODE_DIGITAL_PULL_UP
```

- Enable Step/Dir

```
odrv0.axis0.config.enable_step_dir = True
```

Note that to exit pulse/direction control, you must set `enable_step_dir` to False, enter idle state, and then return to closed-loop control.

- Enable Circular Setpoint (Optional)

To address accuracy drift when handling large target positions, ODrive provides circular setpoint support. This mode keeps floating-point errors within manageable limits. If the motor is commanded beyond the circular range, the position setpoint automatically wraps back to remain within bounds.

Enable the controller's circular setpoint mode

```
odrv0.axis0.controller.config.circular_setpoints = True
```

Set the circular position range (in turns). The setpoint value will be constrained within this range from 0 to the specified limit. Beyond this limit,

will wrap back.

```
# 128 turns form one closed loop
odrv0.axis0.controller.config.circular_setpoint_range = 128.0
```

Determines the number of STEP pulses required within the circular position range, defining the displacement per step

```
# 1024 steps per revolution
odrv0.axis0.controller.config.steps_per_circular_range = 128 * 1024
```

➤ Save configuration (optional)

```
odrv0.save_configuration()
```

After completing the above configuration, enter closed-loop mode in **position** mode to enable PWM control.

4.6 SDK

4.6.1 C/C++ SDK

Based on the CAN Simple protocol, the following C/C++ SDK can be downloaded for integration:
<https://gitee.com/cyberbeast/mwmotorsdk>. Please refer to the detailed test examples on the aforementioned open-source website for integration testing.

4.6.2 Python SDK

First, follow the steps in Section 3.1 to install odrive tool (pip install --upgrade odrive). Refer to Section 3.1.8 for all commands described in that subsection to facilitate Python development.

Below are three examples:

1) Practical Application: Power-On Calibration

```
import odrive import time

odrv0 = odrive.find_any() odrive.utils.dump_errors(odrv0)
odrv0.clear_errors()
odrv0.axis0.requested_state = odrive.utils.AxisState.MOTOR_CALIBRATION time.sleep(5)
while (odrv0.axis0.current_state != 1):
    time.sleep(0.5) odrive.utils.dump_errors(odrv0)
odrv0.axis0.requested_state = odrive.utils.AxisState.ENCODER_OFFSET_CALIBRATION
time.sleep(6)
while (odrv0.axis0.current_state != 1): time.sleep(0.5)
odrive.utils.dump_errors(odrv0) odrv0.axis0.motor.config.pre_calibrated=1
odrv0.axis0.encoder.config.pre_calibrated=1 odrv0.save_configuration()
```

2) Practical Application: Speed Control

```
import odrive import time

odrv0 = odrive.find_any() odrv0.axis0.controller.config.control_mode =
odrive.utils.ControlMode.VELOCITY_CONTROL
odrv0.axis0.controller.config.input_mode = odrive.utils.InputMode.VEL_RAMP
odrv0.axis0.controller.config.vel_ramp_rate=50
odrv0.axis0.requested_state=odrive.utils.AxisState.CLOSED_LOOP_CONTROL odrv0.axis0.controller.input_vel=15
odrive.utils.dump_errors(odrv0) time.sleep(5)
odrv0.axis0.controller.input_vel=0
```

3) Practical Application: Position Control

```
import odrive

odrv0 = odrive.find_any() odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.POSITION_CONTROL
o drv0.axis0.controller.config.input_mode = odrive.utils.InputMode.POS_FILLETTER
odrv0.axis0.requested_state = odrive.utils.AxisState.CLOSED_LOOP_CONTROL odrv0.axis0.controller.input_pos = 10
```

4) Practical Application: Data Acquisition

During R&D and integration, users often need to capture motor operation data—such as voltage/current changes, position/velocity variations, etc. The Python SDK integrates robust data capture capabilities, enabling massive operational data collection with simple scripts to streamline development and integration.

The following code builds upon the previous position control example by adding real-time position and current data capture functionality, saving the data to a CSV file.

```
import odrive import numpy
as np

odrv0 = odrive.find_any() cap =
odrive.utils.BulkCapture(lambda:[odrv0.axis0.motor.current_control.Iq_measured,
odrv0.axis0.encoder.pos_estimate], data_rate=500, duration=2.5)
odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.POSITION_CONTROL
odrv0.axis0.controller.config.input_mode = odrive.utils.InputMode.POS_FI LTER
odrv0.axis0.requested_state = odrive.utils.AxisState.CLOSED_LOOP_CONTROL odrv0.axis0.controller.input_pos = 10
np.savetxt("d:/test.csv", cap.data, delimiter=',')
```

In the BulkCapture statement, `data\_rate` represents the sampling frequency in Hz, `duration` represents the sampling time in seconds, and the lambda expression can incorporate any mathematical operations to facilitate data analysis.

4.6.3 Arduino SDK

Users can control motors via the CAN bus using Arduino, with the underlying protocol described in Section 4.1. Compatible hardware/

Compatible hardware/libraries:

- ✓ Arduino boards with built-in CAN interfaces, such as Arduino UNO R4 Minima, Arduino UNO R4 WIFI, etc.
- ✓ Teensy development boards with built-in CAN interfaces can be accessed using the compatible FlexCAN_T4 library (Teensy 4.0 and Teensy 4.1)
- ✓ Other Arduino-compatible boards can use MCP2515-based CAN expansion boards for access

Below is an example demonstrating how to configure the motor to respond to position control commands from Arduino:

➤ Configuring the Motor

In addition to the basic configuration in Section 3.1.3, configure the control bandwidth to 20 rad/s (the Arduino Uno has limited transmission speed, so the control bandwidth does not need to be too high; if using a faster Arduino, this bandwidth value can be increased):

➤ Configuring CAN

Configure CAN as follows:

➤ Install the ODriveArduino Library

Install the ODriveArduino library following these steps (assuming the Arduino IDE is already installed):

- 1) Open the Arduino IDE

- 2) Sketch -> Include Library -> Manage Libraries
 - 3) Search for "ODriveArduino"
 - 4) Click the found ODriveArduino library to install it
- Arduino Source Code

```
#include <Arduino.h> #include
"ODriveCAN.h"

// Documentation for this example can be found here:
// https://docs.odriverobotics.com/v/latest/guides/arduino-can-guide.html

/* Configuration of example sketch */

// CAN bus baud rate. Ensure this matches for every device on the bus #define CAN_BAUDRATE 500000

// ODrive node_id for odrv0 #define
ODRV0_NODE_ID 0

// Uncomment the line below that corresponds to your hardware.
// See also "Board-specific settings" to adapt the details for your hardware setup.

// #define IS_TEENSY_BUILTIN // Teensy boards with built-in CAN interface (e.g. Teensy 4.1). See below to select which
// interface to use.
// #define IS_ARDUINO_BUILTIN // Arduino boards with built-in CAN interface (e.g. Arduino Uno R4 Minima)
// #define IS_MCP2515 // Any board with external MCP2515-based extension module. See below to configure the module.

/* Board-specific includes */

#if defined(IS_TEENSY_BUILTIN) + defined(IS_ARDUINO_BUILTIN) + defined(IS_MCP2515) != 1
#warning "Select exactly one hardware option at the top of this file."

#if CAN_HOWMANY > 0 || CANFD_HOWMANY > 0
#define IS_ARDUINO_BUILTIN
#warning "guessing that this uses HardwareCAN" #else
#error "Unable to determine hardware version"

-----
```

```
#endif #endif

#ifdef IS_ARDUINO_BUILTIN
// See https://github.com/arduino/ArduinoCore-API/blob/master/api/HardwareCAN.h
// and https://github.com/arduino/ArduinoCore-
renesas/tree/main/libraries/Arduino_CAN

#include <Arduino_CAN.h> #include
<ODriveHardwareCAN.hpp> #endif //
IS_ARDUINO_BUILTIN

#ifdef IS_MCP2515
// See https://github.com/sandeepmistry/arduino-CAN/ #include "MCP2515.h"
#include "ODriveMCP2515.hpp" #endif //
IS_MCP2515

#ifdef IS_TEENSY_BUILTIN
// See https://github.com/tonton81/FlexCAN_T4
// clone https://github.com/tonton81/FlexCAN_T4.git into /src #include <FlexCAN_T4.h>
#include "ODriveFlexCAN.hpp"
struct ODriveStatus; // hack to prevent Teensy compile error #endif // IS_TEENSY_BUILTIN

/* Board-specific settings */

/* Teensy */

#ifdef IS_TEENSY_BUILTIN

FlexCAN_T4<CAN1, RX_SIZE_256, TX_SIZE_16> can_intf;

bool setupCan() { can_intf.begin();
    can_intf.setBaudRate(CAN_BAUDRATE);

    can_intf.setMaxMB(16);

    can_intf.enableFIFO();
    -----
```

```
    can_intf.enableFIFOInterrupt();
    can_intf.onReceive(onCanMessage); return true;
}

#endif // IS_TEENSY_BUILTIN

/* MCP2515-based extension modules -*/ #ifndef IS_MCP2515

MCP2515Class& can_intf = CAN;

// chip select pin used for the MCP2515 #define MCP2515_CS
10

// interrupt pin used for the MCP2515
// NOTE: Not all Arduino pins are interrupt-capable. Check the documentation for your board!
#define MCP2515_INT 2

// Frequency of the crystal oscillator on the MCP2515 breakout board.
// Common values are: 16 MHz, 12 MHz, 8 MHz #define MCP2515_CLK_HZ
8000000

static inline void receiveCallback(int packet_size) { if (packet_size > 8) {
    return; // not supported
}
    CanMsg msg = {.id = (unsigned int)CAN.packetId(), .len = (uint8_t)packet_size}; CAN.readBytes(msg.buffer, packet_size);
    onCanMessage(msg);
}

bool setupCan() {
    // Configure and initialize the CAN bus interface CAN.setPins(MCP2515_CS, MCP2515_INT);
    CAN.setClockFrequency(MCP2515_CLK_HZ); if
    (!CAN.begin(CAN_BAUDRATE)) {
        return false;
    }
}
```

```

    CAN.onReceive(receiveCallback); return true;
}

#endif // IS_MCP2515

/* Arduinos with built-in CAN */ #ifdef

IS_ARDUINO_BUILTIN

HardwareCAN& can_intf = CAN;

bool setupCan() {
    return can_intf.begin((CanBitRate)CAN_BAUDRATE);
}

#endif

/* Example sketch */

// Instantiate ODrive objects
ODriveCAN odrv0(wrap_can_intf(can_intf), ODRV0_NODE_ID); // Standard CAN message ID
ODriveCAN* odrives[] = {&odrv0}; // Ensure all ODriveCAN instances are accounted for here

struct ODriveUserData { Heartbeat_msg_t-----
    last_heartbeat;    bool received_heartbeat = false;
    Get_Encoder_Estimates_msg_t last_feedback; bool received_feedback
    = false;
};

// Maintain application-specific user data for each ODrive. ODriveUserData odrv0_user_data;

// Called every time a Heartbeat message arrives from the ODrive void onHeartbeat(Heartbeat_msg_t&
msg, void* user_data) {
    ODriveUserData* odrv_user_data = static_cast<ODriveUserData*>(user_data); odrv_user_data->last_heartbeat = msg;
    odrv_user_data->received_heartbeat = true;
}

```

```
// Called every time a feedback message arrives from the ODrive
void onFeedback(Get_Encoder_Estimates_msg_t& msg, void* user_data) {  ODriveUserData* odrv_user_data =
    static_cast<ODriveUserData*>(user_data);  odrv_user_data->last_feedback = msg;
    odrv_user_data->received_feedback = true;
}

// Called for every message that arrives on the CAN bus
void onCanMessage(const CanMsg& msg) {
    for (auto odrive: odrives) { onReceive(msg,
        *odrive);
    }
}

void setup() { Serial.begin(115200);

    // Wait for up to 3 seconds for the serial port to be opened on the PC side.
    // If no PC connects, continue anyway. for (int i = 0; i < 30 &&
    !Serial; ++i) {
        delay(100);
    }
    delay(200);

    Serial.println("Starting ODriveCAN demo");

    // Register callbacks for the heartbeat and encoder feedback messages odrv0.onFeedback(onFeedback,
    &odrv0_user_data); odrv0.onStatus(onHeartbeat, &odrv0_user_data);

    // Configure and initialize the CAN bus interface. This function depends on
    // your hardware and the CAN stack you're using. if (!setupCan()) {
        Serial.println("CAN failed to initialize: reset required"); while (true); // spin
        indefinitely
    }

    Serial.println("Waiting for ODrive..."); while
    (!odrv0_user_data.received_heartbeat) {
        pumpEvents(can_intf); delay(100);
    }
}
```

```
Serial.println("found ODrive");

// Request bus voltage and current (1-second timeout) Serial.println("Attempting to read bus voltage and
current"); Get_Bus_Voltage_Current_msg_t vbus;
if (!odrv0.request(vbus, 1)) { Serial.println("vbus request
failed!"); while (true); // spin indefinitely
}

Serial.print("DC voltage [V]: ");
Serial.println(vbus.Bus_Voltage); Serial.print("DC
current [A]: "); Serial.println(vbus.Bus_Current);

Serial.println("Enabling closed loop control..."); while
(odrv0_user_data.last_heartbeat.Axis_State !=
ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL) { odrv0.clearErrors();
delay(1); odrv0.setState(ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL);

// Pump events for 150ms. This delay is needed for two reasons;
// 1. If an error condition occurs, such as missing DC power, the ODrive may
// briefly attempt to enter CLOSED_LOOP_CONTROL state, so we cannot rely
// on the first heartbeat response, so we want to receive at least two
// heartbeats (default interval 100ms).
// 2. If the bus is congested, the setState command won't get through
// immediately but may be delayed. for (int i = 0;
i < 15; ++i) {
delay(10); pumpEvents(can_intf);
}
}

Serial.println("ODrive running!");
}

void loop() {
pumpEvents(can_intf); // This is required on some platforms to handle incoming feedback CAN messages
```

```
float SINE_PERIOD = 2.0f; // Period of the position command sine wave in seconds

float t = 0.001 * millis();

float phase = t * (TWO_PI / SINE_PERIOD);

odrv0.setPosition( sin(phase), //
    position
    cos(phase) * (2π / SINE_PERIOD) // velocity feedforward (optional)
);

// print position and velocity for Serial Plotter if
(odrv0_user_data.received_feedback) {
    Get_Encoder_Estimates_msg_t feedback = odrv0_user_data.last_feedback; odrv0_user_data.received_feedback = false;
    Serial.print("odrv0-pos:");
    Serial.print(feedback.Pos_Estimate); Serial.print(",");
    Serial.print("odrv0-vel:");
    Serial.println(feedback.Vel_Estimate);
}
}
```

4.6.4 ROS SDK

The following steps have been tested and verified on Ubuntu 23.04 and ROS2 Iron. However, they are not supported on MAC or Windows platforms and have not been validated on other ROS2 versions. They may require adjustments before use.

4.6.4.1 Install the odrive_can package

1. Create a new ROS2 workspace (see <https://docs.ros.org/en/iron/index.html>)
2. Clone the code into the src directory of the above workspace directory using git clone https://github.com/odriverobotics/odrive_can
3. In the terminal, navigate to the root directory of the workspace and run:

```
colcon build --packages-select odrive_can
```

4. Pre-run environment setup:

```
source ./install/setup.bash
```

5. Run the example node:

```
ros2 launch odrive_can example_launch.yaml
```

4.6.4.2 Invoke services and view messages

Assuming the above odrive_can_node runs in the odrive_axis0 namespace (configurable set in ./launch/example_launch.yaml). Once the node in section 4.4.4.1 is running, you can view the published topic messages, such as:

```
ros2 topic echo /odrive_axis0/controller_status ros2 topic echo /odrive_axis0/odrive_status
```

Call the open service interface. The following call initiates motor calibration:

```
ros2 service call /odrive_axis0/request_axis_state /odrive_can/srv/AxisState "{axis_requested_state: 4}"
```

5 Frequently Asked Questions and Error Codes (To be updated)

5.6 Frequently Asked Questions (FAQ)

5.7 Error Codes

Error Category	Error Code	odrivetool Display	Description
System Exception	0x00000002	DC_BUS_UNDER_VOLTAGE	Power supply voltage too low
	0x00000004	DC_BUS_OVER_VOLTAGE	Power supply voltage too high
	0x00000008	DC_BUS_OVER_REGEN_CURRENT	DC Bus Over-Regeneration Current High
	0x00000010	DC_BUS_OVER_CURRENT	Power supply forward (discharge) current over High
Drive Abnormal	0x00000001	INVALID_STATE	Driver state error
	0x00000040	MOTOR_FAILED	Motor failure
	0x00000100	ENCODER_FAILED	Encoder Failure
	0x00000200	CONTROLLER_FAILED	Controller Failure
	0x00001000	MIN_ENDSTOP_PRESSED	Minimum End-of-Travel Triggered
	0x00002000	MAX_ENDSTOP_PRESSED	High limit switch triggered
	0x00004000	ESTOP_REQUESTED	Emergency Stop
	0x00020000	HOMING_WITHOUT_ENDSTOP	Homing without endstop
	0x00080000	UNKNOWN_POSITION	No position information
	0x00000001	PHASE_RESISTANCE_OUT_OF_RANGE	Phase-to-phase resistance exceeds normal range
Motor Abnor	0x00000002	PHASE_INDUCTANCE_OUT_OF_RANGE	Phase-to-phase inductance exceeds normal range

mal	0x00000010	CONTROL_DEADLINE_MISSED	FOC frequency too high
	0x00000080	MODULATION_MAGNITUDE	SVM modulation error
	0x00000400	CURRENT_SENSE_SATURATION	Phase Current Saturation
	0x00001000	CURRENT_LIMIT_VIOLATION	Motor Current Exceeds Limit
	0x00020000	MOTOR_THERMISTOR_OVER_TEMP	Motor temperature too high
	0x00040000	FET Thermistor Over-Temperature	Driver Overheating
	0x00080000	TIMER_UPDATE_MISSED	FOC processing delayed
	0x00100000	CURRENT_MEASUREMENT_UNAVAILABLE	Phase current sampling lost
	0x00200000	CONTROLLER_FAILED	Control Abnormal
	0x00400000	I_BUS_OUT_OF_RANGE	Bus Current Out of Range
	0x00800000	BRAKE_RESISTOR_DISARMED	Brake Resistor Drive Abnormal
	0x01000000	SYSTEM_LEVEL	System-level fault
	0x02000000	BAD_TIMING	Bad Timing
	0x04000000	UNKNOWN_PHASE_ESTIMATE	Motor position unknown
	0x08000000	UNKNOWN_PHASE_VEL	Motor speed unknown
	0x10000000	UNKNOWN_TORQUE	Unknown Torque
	0x20000000	UNKNOWN_CURRENT_COMMAND	Unknown Torque Control
	0x40000000	UNKNOWN_CURRENT_MEASUREMENT	Unknown current measurement value
	0x80000000	UNKNOWN_VBUS_VOLTAGE	Unknown voltage sample value
	0x100000000	UNKNOWN_VOLTAGE_COMMAND	Unknown voltage command
	0x200000000	UNKNOWN_GAINS	Unknown Current Loop Gains
	0x400000000	CONTROLLER_INITIALIZING	Controller Initialization Error
	0x800000000	UNBALANCED_PHASES	Three-phase imbalance
Control Abnormal	0x00000001	OVERSPEED	Over-Speed
	0x00000002	INVALID_INPUT_MODE	Invalid control input mode
	0x00000004	UNSTABLE_GAIN	Phase-locked loop gain unstable
	0x00000020	INVALID_ESTIMATE	Position/velocity unstable
	0x00000080	SPIN_OUT_DETECTED	Mechanical power and electrical power mismatch (encoder calibration incorrect, or unstable magnet)
Encoder Abnormal	0x00000001	UNSTABLE_GAIN	Encoder bandwidth too high
	0x00000002	CPR_POLEPAIRE_MISMATCH	CPR and pole pair mismatch
	0x00000004	NO_RESPONSE	Encoder no response
	0x00000400	SEC_ENC_COM_FAIL	Second Encoder Communication Error